RELATIONAL DATABASE PROJECT · MYSQL 8

Hotel Booking & Revenue Optimization

A normalised MySQL database for multi-property hotel operations — guests, bookings, rooms, payments, cancellations, reviews, services, staff and expenses — with an analytics layer of fifteen CTE-based reporting questions built on joins, window functions and CASE-driven segmentation.

₹327,500	30	4.22	16.67%
COMPLETED REVENUE	BOOKINGS	AVG GUEST RATING	CANCELLATION RATE

DATABASE ENGINE

MySQL 8.0

InnoDB (default engine)

SCHEMA

hotel_analytics

15 tables · 15 analytical queries · 15 foreign keys

SEED DATA

239 rows

20 customers · 30 bookings · 5 hotels

CONCEPTS COVERED

DDL · DML · Joins

CTEs, LAG(), DENSE_RANK(), CASE, ALTER TABLE

WHAT THIS DOCUMENT CONTAINS

Every object in the project is reproduced here in full: the complete DDL for all fifteen tables, every INSERT statement, the executed result sets of all verification checks, fifteen analytical questions with their SQL, output and charts, and a code-review section documenting the defects found in the original script together with the correction for each.

CONTENTS

Table of Contents

01 **Project Overview** 3
Scope, objectives and the engineering skills the schema demonstrates.

02 **Schema Architecture** 5
Entity map, relationship model and the referential integrity plan.

03 **Database & Table Definitions** 7
Steps 1–17: DDL for all fifteen tables.

04 **Master & Reference Data** 12
Steps 18–23: hotels, room types, sources, services, campaigns, guests.

05 **Operational Data Load** 15
Steps 24–35: rooms, staff, bookings, payments, reviews, expenses and checks.

06 **Hotel Performance Analytics** 27
Questions 1–5: KPI report, profitability, revenue trend, sources, customer value.

07 **Operations, Risk & Guest Analytics** 44
Questions 6–15: room types, cancellations, reviews, costs, lead time, collections.

08 **Code Review & Corrections** 57
Six data and logic defects, six design observations, and the fix for each.

09 **Results, Setup & Roadmap** 60
How to run the project, object inventory and future enhancements.

SECTION 01

Project Overview

A single MySQL schema that models the booking-to-payment lifecycle of a five-property hotel group, then layers revenue, risk and guest analytics on top of it.

The `hotel_analytics` database models a hotel group operating five properties across India — Mumbai, Goa, Delhi, Manali and Bengaluru. It tracks the full guest lifecycle: registration, booking through one of five channels, room assignment, payment, cancellation and refund, in-stay services and the post-stay review — alongside the cost side of the business: staff and operating expenses and marketing campaigns. On top of that transactional core sit fifteen analytical questions, each answered with a chain of common table expressions, that cover the questions a revenue manager or general manager would actually ask.

BASE TABLES	FOREIGN KEYS	ANALYTICAL QUERIES
15	**15**	**15**
master, core, activity	declared in DDL	CTE-based

SEEDED ROWS	HOTELS	BOOKINGS
239	**5**	**30**
across 15 tables	Mumbai, Goa, Delhi, Manali, Bengaluru	Jan–May 2025

Objectives

- **Normalised schema.** Every business entity — hotel, room type, room, guest, booking, payment, cancellation, review, service, employee, expense, campaign, booking source — owns its own table and is linked through declared foreign keys rather than duplicated columns.
- **Constraint-level data quality.** Primary keys, a UNIQUE guest email, a composite UNIQUE (hotel_id, room_number) so a room number can repeat across hotels but never within one, NOT NULL on every link column, and DEFAULT values on discounts, refunds and quantities.
- **Realistic seed data.** Thirty bookings across five months, with payments, refunds, reviews, service usage and expenses, so every query returns meaningful output instead of an empty set.
- **Answer business questions in SQL.** Hotel KPI report, profitability and cancellation risk, month-over-month growth, channel and commission performance, customer lifetime value, room-type demand, review sentiment, cost efficiency, lead-time behaviour, utilisation and collections.
- **Review its own work.** Section 08 documents the defects found by checking the seeded data against the business rules the schema implies.

Functional Modules

Module	Tables	Responsibility
Property Network	`hotels, rooms, room_types`	Hotel identity, star rating and type; physical rooms per hotel and their category, base price and capacity.
Guest Register	`customers`	Guest profile, home city and Regular / Premium tier.
Reservations	`bookings, booking_rooms, booking_sources`	Stay dates, guests, status, amounts and discount; the room(s) and nightly rate assigned; the channel and its commission.
Payments	`payments, cancellations`	Card, UPI and cash receipts with status; cancellation reason and refund amount.
Guest Experience	`reviews, services, service_usage`	Post-stay rating and text; the add-on service catalogue (Food, Wellness, Transport, Housekeeping) and what each booking consumed.
Workforce & Cost	`employees, expenses`	Staff posting, role and salary; monthly utilities, maintenance, staff and marketing spend by hotel.
Marketing	`marketing_campaigns`	Campaign name, channel, run dates and budget.

Technology and Techniques

Area	Applied in this project
DDL	`CREATE DATABASE`, `CREATE TABLE`, `ALTER TABLE ... ADD CONSTRAINT ... FOREIGN KEY`
Constraints	`AUTO_INCREMENT` keys, `UNIQUE`, composite `UNIQUE (hotel_id, room_number)`, `NOT NULL`, `DEFAULT 0` and `DEFAULT 1`
DML	Multi-row `INSERT` loads for all fifteen tables
Joins	`INNER JOIN`, `LEFT JOIN`, `CROSS JOIN` against a one-row reporting-period CTE, four-table room-to-hotel chains
CTE pipelines	`WITH` chains of up to 21 named steps (Question 1) that are merged in a final KPI select
Aggregation	`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`, `COUNT(DISTINCT ...)`, conditional aggregation with `SUM(CASE WHEN ...)`, `GROUP_CONCAT`
Window functions	`LAG()` for month-over-month growth, `ROW_NUMBER()` to pick the latest month, `DENSE_RANK()` league tables
Date logic	`DATEDIFF` for length of stay and lead time, `DATE_FORMAT` for monthly buckets
Null-safe maths	`COALESCE`, `NULLIF` in every ratio so an empty denominator returns NULL instead of an error
Segmentation	`CASE` expressions that turn numbers into labels: risk class, value tier, utilisation band, recommended action
Data quality	Invalid date-range check and orphan-booking check run after the load

SECTION

02 Schema Architecture

Fifteen tables arranged in three tiers: master entities, the reservation core, and the activity tables that record what actually happened.

The schema is deliberately shallow and wide. Five master entities — `hotels`, `customers`, `room_types`, `booking_sources` and `services` — plus the standalone `marketing_campaigns` own no foreign keys and can be loaded first. Everything else hangs off them. `bookings` is the hinge of the model: it references the guest, the hotel and the channel, and five activity tables (rooms assigned, payments, cancellations, reviews, service usage) all reference it.

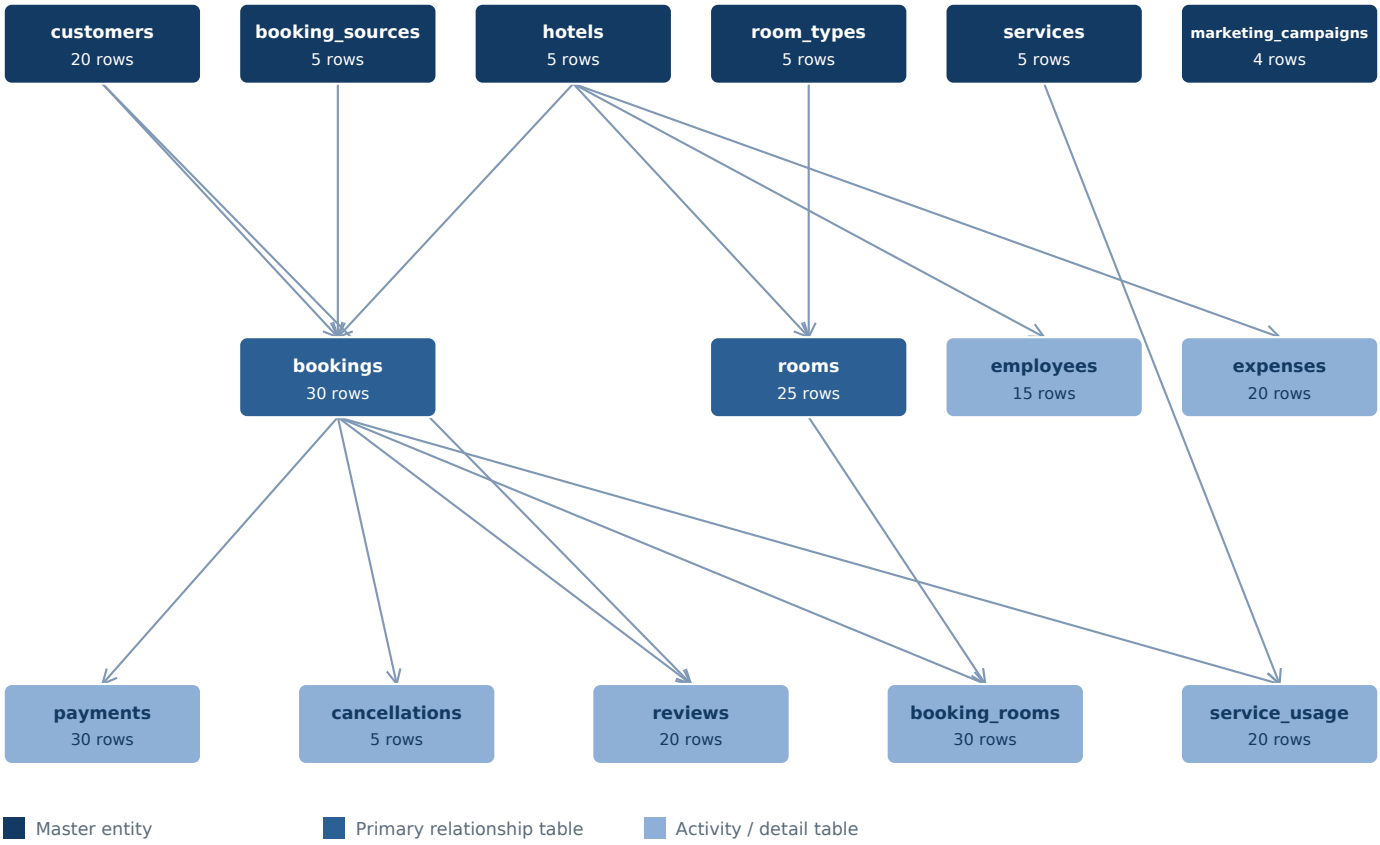


Figure 2.1 — Entity relationship map of the hotel_analytics schema. Arrows point from the parent table to the child table holding the foreign key.

Referential Integrity Map

Fifteen foreign keys bind the schema together. Two details are worth noting: `bookings` carries both a hotel and a room link (via `booking_rooms`), so a booking can in principle span several rooms; and `reviews` references both the booking and the customer, so a review is always attributable to a stay and to a guest.

Child table	Foreign key column(s)	References
rooms	hotel_id, room_type_id	hotels, room_types
bookings	customer_id, hotel_id, booking_source_id	customers, hotels, booking_sources (source added by ALTER TABLE)
booking_rooms	booking_id, room_id	bookings, rooms
payments	booking_id	bookings
cancellations	booking_id	bookings
reviews	booking_id, customer_id	bookings, customers
service_usage	booking_id, service_id	bookings, services
employees	hotel_id	hotels
expenses	hotel_id	hotels

Build Order

Because foreign keys are declared inline at creation time, tables must be created in dependency order. The one exception is `bookings.booking_source_id`, which is declared as a plain column and wired to `booking_sources` later with `ALTER TABLE` — a deliberate demonstration of evolving a schema in place.

Order	Objects created	Why here
Tier 1	`customers, hotels, room_types, booking_sources, services, marketing_campaigns`	No outbound dependencies; safe to create and load first.
Tier 2	`rooms, bookings, employees, expenses`	Depend only on Tier 1 master entities.
Tier 3	`booking_rooms, payments, cancellations, reviews, service_usage`	Depend on bookings (and rooms or services); these hold the activity history.
Tier 4	`15 CTE-based analytical queries`	Read-only layer; run once all base tables are populated.

Design Conventions

- **Surrogate keys everywhere.** Every table uses an `INT AUTO_INCREMENT` primary key, keeping joins narrow and stable.
- **Money as DECIMAL.** Amounts are `DECIMAL(10,2)` / `DECIMAL(12,2)`; commission is `DECIMAL(5,2)`; floating point is never used for currency.
- **Natural keys kept unique where it matters.** `customers.email` is unique, and `(hotel_id, room_number)` is a composite unique key.
- **Status as text.** Booking, payment, room and employment state are `VARCHAR` columns (Completed / Cancelled, Paid / Pending / Refunded), which keeps history queryable without deletes.
- **Derived measures are computed, not stored.** Net revenue, length of stay, lead time and occupancy are calculated inside the queries from stored dates and amounts.

SECTION

03 Database and Table Definitions

Steps 1–17 of the script: creating the schema and all fifteen base tables, then verifying them.

The statements below are reproduced exactly as executed, with consistent indentation added for reading. Each table is preceded by a short note on the columns that carry business meaning.

Step 1 — Create and select the schema

STEP 1 — SCHEMA CREATION

```
CREATE DATABASE hotel_analytics;
USE hotel_analytics;

-- Verify selected database
SELECT DATABASE();
```

Step 2 — customers

Guest identity and tier. `email` is unique so a guest cannot be registered twice; `customer_type` carries the Regular / Premium tier used in the guest analysis.

STEP 2 — CREATE TABLE CUSTOMERS

```
CREATE TABLE customers (
  customer_id INT PRIMARY KEY AUTO_INCREMENT,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50),
  email VARCHAR(100) UNIQUE,
  phone VARCHAR(20),
  gender VARCHAR(20),
  country VARCHAR(50),
  city VARCHAR(50),
  registration_date DATE,
  customer_type VARCHAR(30)
)
```

Step 3 — hotels

The property master. `star_rating` is `DECIMAL(2,1)` so half-stars are representable, and `total_rooms` records the advertised inventory.

STEP 3 — CREATE TABLE HOTELS

```
CREATE TABLE hotels (
  hotel_id INT PRIMARY KEY AUTO_INCREMENT,
  hotel_name VARCHAR(100) NOT NULL,
  city VARCHAR(50),
  country VARCHAR(50),
  star_rating DECIMAL(2,1),
  hotel_type VARCHAR(50),
  total_rooms INT,
  opening_date DATE
)
```

Step 4 — room_types

Five room categories with a list `base_price` and a `max_guests` capacity. Actual nightly rates are stored per booking, so list price and charged price can differ.

STEP 4 — CREATE TABLE ROOM TYPES

```
CREATE TABLE room_types (
  room_type_id INT PRIMARY KEY AUTO_INCREMENT,
  room_type_name VARCHAR(50) NOT NULL,
  base_price DECIMAL(10,2),
  max_guests INT,
  description VARCHAR(255)
)
```

Step 5 — rooms

Physical rooms. The composite `UNIQUE (hotel_id, room_number)` lets room 101 exist in every hotel but only once in each.

STEP 5 — CREATE TABLE ROOMS

```
CREATE TABLE rooms (
  room_id INT PRIMARY KEY AUTO_INCREMENT,
  hotel_id INT NOT NULL,
  room_type_id INT NOT NULL,
  room_number VARCHAR(10) NOT NULL,
  floor_number INT,
  room_status VARCHAR(30),
  FOREIGN KEY (hotel_id) REFERENCES hotels(hotel_id),
  FOREIGN KEY (room_type_id) REFERENCES room_types(room_type_id),
  UNIQUE (hotel_id, room_number)
)
```

Step 6 — bookings

The hinge of the schema. Stores guest, hotel, dates, party size, status, gross `total_amount` and `discount_amount` (default 0). `booking_source_id` is wired up later with `ALTER TABLE`.

STEP 6 — CREATE TABLE BOOKINGS

```
CREATE TABLE bookings (
  booking_id INT PRIMARY KEY AUTO_INCREMENT,
  customer_id INT NOT NULL,
  hotel_id INT NOT NULL,
  booking_source_id INT,
  booking_date DATE NOT NULL,
  check_in_date DATE NOT NULL,
  check_out_date DATE NOT NULL,
  number_of_guests INT,
  booking_status VARCHAR(30),
  total_amount DECIMAL(12,2),
  discount_amount DECIMAL(12,2) DEFAULT 0,
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
  FOREIGN KEY (hotel_id) REFERENCES hotels(hotel_id)
)
```

Step 7 — booking_rooms

Links a booking to the room(s) it occupies and records the `nightly_rate` actually charged.

STEP 7 — CREATE TABLE BOOKING_ROOMS

```
CREATE TABLE booking_rooms (
  booking_room_id INT PRIMARY KEY AUTO_INCREMENT,
  booking_id INT NOT NULL,
  room_id INT NOT NULL,
  nightly_rate DECIMAL(10,2),
  FOREIGN KEY (booking_id) REFERENCES bookings(booking_id),
  FOREIGN KEY (room_id) REFERENCES rooms(room_id)
)
```

Step 8 — payments

One row per payment event with method, amount and status (Paid, Pending, Refunded) plus the gateway `transaction_reference`.

STEP 8 — CREATE TABLE PAYMENTS

```
CREATE TABLE payments (
  payment_id INT PRIMARY KEY AUTO_INCREMENT,
  booking_id INT NOT NULL,
  payment_date DATE,
  payment_method VARCHAR(30),
  payment_amount DECIMAL(12,2),
  payment_status VARCHAR(30),
  transaction_reference VARCHAR(100),
  FOREIGN KEY (booking_id) REFERENCES bookings(booking_id)
)
```

Step 9 — cancellations

Reason and `refund_amount` (default 0) for each cancelled booking.

STEP 9 — CREATE TABLE CANCELLATIONS

```
CREATE TABLE cancellations (
  cancellation_id INT PRIMARY KEY AUTO_INCREMENT,
  booking_id INT NOT NULL,
  cancellation_date DATE,
  cancellation_reason VARCHAR(255),
  refund_amount DECIMAL(12,2) DEFAULT 0,
  FOREIGN KEY (booking_id) REFERENCES bookings(booking_id)
)
```

Step 10 — reviews

Rating (`DECIMAL(2,1)`) and free text, tied to both the booking and the guest.

STEP 10 — CREATE TABLE REVIEWS

```sql
CREATE TABLE reviews (
  review_id INT PRIMARY KEY AUTO_INCREMENT,
  booking_id INT NOT NULL,
  customer_id INT NOT NULL,
  rating DECIMAL(2,1),
  review_date DATE,
  review_text VARCHAR(500),
  FOREIGN KEY (booking_id) REFERENCES bookings(booking_id),
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
)
```

Step 11 — services

The add-on catalogue — name, category and list price.

STEP 11 — CREATE TABLE SERVICES

```sql
CREATE TABLE services (
  service_id INT PRIMARY KEY AUTO_INCREMENT,
  service_name VARCHAR(100) NOT NULL,
  service_category VARCHAR(50),
  service_price DECIMAL(10,2)
)
```

Step 12 — service_usage

What a booking consumed: service, date, `quantity` (default 1) and `total_cost`.

STEP 12 — CREATE TABLE SERVICE USAGE

```sql
CREATE TABLE service_usage (
  usage_id INT PRIMARY KEY AUTO_INCREMENT,
  booking_id INT NOT NULL,
  service_id INT NOT NULL,
  usage_date DATE,
  quantity INT DEFAULT 1,
  total_cost DECIMAL(10,2),
  FOREIGN KEY (booking_id) REFERENCES bookings(booking_id),
  FOREIGN KEY (service_id) REFERENCES services(service_id)
)
```

Step 13 — employees

Staff per hotel with role, department, salary and status.

STEP 13 — CREATE TABLE EMPLOYEES

```sql
CREATE TABLE employees (
  employee_id INT PRIMARY KEY AUTO_INCREMENT,
  hotel_id INT NOT NULL,
  employee_name VARCHAR(100),
  job_title VARCHAR(50),
  department VARCHAR(50),
  salary DECIMAL(12,2),
  hire_date DATE,
  employment_status VARCHAR(30),
  FOREIGN KEY (hotel_id) REFERENCES hotels(hotel_id)
)
```

Step 14 — expenses

Operating cost postings per hotel, classified into Utilities, Maintenance, Staff and Marketing.

STEP 14 — CREATE TABLE EXPENSES

```sql
CREATE TABLE expenses (
  expense_id INT PRIMARY KEY AUTO_INCREMENT,
  hotel_id INT NOT NULL,
  expense_date DATE,
  expense_category VARCHAR(50),
  expense_description VARCHAR(255),
  expense_amount DECIMAL(12,2),
  FOREIGN KEY (hotel_id) REFERENCES hotels(hotel_id)
)
```

Step 15 — marketing_campaigns

Campaign name, channel, start and end dates and budget.

STEP 15 — CREATE TABLE MARKETING_CAMPAIGNS

```
CREATE TABLE marketing_campaigns (
  campaign_id INT PRIMARY KEY AUTO_INCREMENT,
  campaign_name VARCHAR(100),
  marketing_channel VARCHAR(50),
  start_date DATE,
  end_date DATE,
  campaign_budget DECIMAL(12,2)
)
```

Step 16 — booking_sources

Booking channels with a `commission_percentage` and a Direct / OTA / Agent category.

STEP 16 — CREATE TABLE BOOKING SOURCES

```
CREATE TABLE booking_sources (
  booking_source_id INT PRIMARY KEY AUTO_INCREMENT,
  source_name VARCHAR(50) NOT NULL,
  commission_percentage DECIMAL(5,2),
  source_category VARCHAR(50)
)
```

Step 17 — Verify the tables

STEP 17 — SHOW TABLES

```
SHOW TABLES;
```

OUTPUT — 15 ROWS (CREATION ORDER)

Tables_in_hotel_analytics
customers
hotels
room_types
rooms
bookings
booking_rooms
payments
cancellations
reviews
services
service_usage
employees
expenses
marketing_campaigns
booking_sources

Table Inventory

Table	Rows	Unique keys	Primary purpose
hotels	5	—	Property master
room_types	5	—	Room categories and list price
booking_sources	5	—	Booking channels and commission
services	5	—	Add-on service catalogue
marketing_campaigns	4	—	Campaign calendar and budget
customers	20	email	Guest register
rooms	25	(hotel_id, room_number)	Physical room inventory
employees	15	—	Staff register
bookings	30	—	Reservation header
booking_rooms	30	—	Room assignment and rate
payments	30	—	Receipts and refunds
cancellations	5	—	Cancellation reason and refund
reviews	20	—	Post-stay ratings
service_usage	20	—	Add-on consumption
expenses	20	—	Operating cost postings

SECTION

04 Master and Reference Data

Steps 18–23: loading the tables that own no foreign keys — hotels, room types, booking channels, services, campaigns and the guest register.

These loads must run first. Every later insert depends on the identifiers generated here, and because each foreign key is enforced, loading out of order is rejected by the engine rather than silently creating orphan rows.

Step 18 — hotels (5 rows)

Five properties from 3 to 5 stars, opened between 2017 and 2021, spanning Luxury, Resort and Business types.

STEP 18 — INSERT INTO HOTELS

```
INSERT INTO hotels
(hotel_name, city, country, star_rating, hotel_type, total_rooms, opening_date)
VALUES
  ('Grand Palace Hotel', 'Mumbai', 'India', 5.0, 'Luxury', 120, '2018-01-15'),
  ('Sea View Resort', 'Goa', 'India', 4.0, 'Resort', 90, '2019-06-10'),
  ('Royal Comfort Inn', 'Delhi', 'India', 3.0, 'Business', 80, '2020-03-20'),
  ('Mountain Escape', 'Manali', 'India', 4.0, 'Resort', 65, '2021-05-12'),
  ('City Business Hotel', 'Bengaluru', 'India', 4.0, 'Business', 100, '2017-11-01')
```

LOADED DATA — HOTELS

hotel_id	hotel_name	city	country	star_rating	hotel_type	total_rooms	opening_date
1	Grand Palace Hotel	Mumbai	India	5	Luxury	120	2018-01-15
2	Sea View Resort	Goa	India	4	Resort	90	2019-06-10
3	Royal Comfort Inn	Delhi	India	3	Business	80	2020-03-20
4	Mountain Escape	Manali	India	4	Resort	65	2021-05-12
5	City Business Hotel	Bengaluru	India	4	Business	100	2017-11-01

Step 19 — room_types (5 rows)

Five categories priced from ₹2,500 (Standard) to ₹8,000 (Suite), with capacity from 2 to 5 guests.

STEP 19 — INSERT INTO ROOM_TYPES

```
INSERT INTO room_types
(room_type_name, base_price, max_guests, description)
VALUES
  ('Standard', 2500, 2, 'Comfortable standard room'),
  ('Deluxe', 4000, 3, 'Premium deluxe room'),
  ('Suite', 8000, 4, 'Luxury suite'),
  ('Family Room', 6000, 5, 'Room for families'),
  ('Executive', 5500, 2, 'Business executive room')
```

LOADED DATA — ROOM_TYPES

room_type_id	room_type_name	base_price	max_guests	description
1	Standard	2500	2	Comfortable standard room
2	Deluxe	4000	3	Premium deluxe room
3	Suite	8000	4	Luxury suite
4	Family Room	6000	5	Room for families
5	Executive	5500	2	Business executive room

Step 20 — booking_sources (5 rows)

Two commission-free direct channels, two OTAs at 15% and 18%, and a travel agent at 10%.

STEP 20 — INSERT INTO BOOKING SOURCES

```
INSERT INTO booking_sources
(source_name, commission_percentage, source_category)
VALUES
  ('Hotel Website', 0.00, 'Direct'),
  ('Mobile App', 0.00, 'Direct'),
  ('Booking.com', 15.00, 'OTA'),
  ('MakeMyTrip', 18.00, 'OTA'),
  ('Travel Agent', 10.00, 'Agent')
```

LOADED DATA — BOOKING_SOURCES

booking_source_id	source_name	commission_percentage	source_category
1	Hotel Website	0	Direct
2	Mobile App	0	Direct
3	Booking.com	15	OTA
4	MakeMyTrip	18	OTA
5	Travel Agent	10	Agent

Step 21 — services (5 rows)

Five add-ons across Food, Wellness, Housekeeping and Transport, priced from ₹300 to ₹2,000.

STEP 21 — INSERT INTO SERVICES

```
INSERT INTO services
(service_name, service_category, service_price)
VALUES
  ('Breakfast', 'Food', 500),
  ('Spa', 'Wellness', 2000),
  ('Laundry', 'Housekeeping', 300),
  ('Airport Transfer', 'Transport', 1500),
  ('Room Service', 'Food', 700)
```

LOADED DATA — SERVICES

service_id	service_name	service_category	service_price
1	Breakfast	Food	500
2	Spa	Wellness	2000
3	Laundry	Housekeeping	300
4	Airport Transfer	Transport	1500
5	Room Service	Food	700

Step 22 — marketing_campaigns (4 rows)

Four campaigns across Social Media, Email, Search Ads and Mobile App, with budgets from ₹75,000 to ₹200,000.

STEP 22 — INSERT INTO MARKETING_CAMPAIGNS

```
INSERT INTO marketing_campaigns
(campaign_name, marketing_channel, start_date, end_date, campaign_budget)
VALUES
  ('Summer Getaway', 'Social Media', '2025-04-01', '2025-06-30', 150000),
  ('Festive Offers', 'Email', '2025-10-01', '2025-11-15', 100000),
  ('Winter Escape', 'Search Ads', '2025-12-01', '2026-02-28', 200000),
  ('Weekend Deals', 'Mobile App', '2026-01-01', '2026-12-31', 75000)
```

LOADED DATA — MARKETING_CAMPAIGNS

campaign_id	campaign_name	marketing_channel	start_date	end_date	campaign_budget
1	Summer Getaway	Social Media	2025-04-01	2025-06-30	150000
2	Festive Offers	Email	2025-10-01	2025-11-15	100000
3	Winter Escape	Search Ads	2025-12-01	2026-02-28	200000
4	Weekend Deals	Mobile App	2026-01-01	2026-12-31	75000

Step 23 — Guest register (20 rows)

Twenty guests from fifteen Indian cities registered between January and June 2025. Phone numbers are synthetic and emails use the reserved `example.com` domain, so no real personal data appears in the portfolio project. Eight are Premium and twelve Regular.

STEP 23 — INSERT INTO CUSTOMERS

```
INSERT INTO customers
(first_name, last_name, email, phone, gender, country, city, registration_date, customer_type)
VALUES
  ('Aarav', 'Shah', 'aarav.shah@example.com', '9000000001', 'Male', 'India', 'Mumbai', '2025-01-05', 'Regular'),
  ('Priya', 'Patel', 'priya.patel@example.com', '9000000002', 'Female', 'India', 'Mumbai', '2025-01-10', 'Premium'),
  ('Rahul', 'Verma', 'rahul.verma@example.com', '9000000003', 'Male', 'India', 'Delhi', '2025-01-15', 'Regular'),
  ('Sneha', 'Rao', 'sneha.rao@example.com', '9000000004', 'Female', 'India', 'Bengaluru', '2025-02-01', 'Premium'),
  ('Kabir', 'Mehta', 'kabir.mehta@example.com', '9000000005', 'Male', 'India', 'Pune', '2025-02-10', 'Regular'),
  ('Ananya', 'Das', 'ananya.das@example.com', '9000000006', 'Female', 'India', 'Kolkata', '2025-02-15', 'Regular'),
  ('Vikram', 'Singh', 'vikram.singh@example.com', '9000000007', 'Male', 'India', 'Jaipur', '2025-03-01', 'Premium'),
  ('Isha', 'Kapoor', 'isha.kapoor@example.com', '9000000008', 'Female', 'India', 'Chandigarh', '2025-03-05', 'Regular'),
  ('Rohan', 'Nair', 'rohan.nair@example.com', '9000000009', 'Male', 'India', 'Kochi', '2025-03-10', 'Premium'),
  ('Meera', 'Joshi', 'meera.joshi@example.com', '9000000010', 'Female', 'India', 'Nashik', '2025-03-15', 'Regular'),
  ('Aditya', 'Gupta', 'aditya.gupta@example.com', '9000000011', 'Male', 'India', 'Lucknow', '2025-04-01', 'Regular'),
  ('Kavya', 'Iyer', 'kavya.iyer@example.com', '9000000012', 'Female', 'India', 'Chennai', '2025-04-05', 'Premium'),
```

```
('Arjun', 'Malhotra', 'arjun.malhotra@example.com', '9000000013', 'Male', 'India', 'Mumbai', '2025-04-10', 'Regular'),
('Neha', 'Kulkarni', 'neha.kulkarni@example.com', '9000000014', 'Female', 'India', 'Pune', '2025-04-15', 'Premium'),
('Dev', 'Khanna', 'dev.khanna@example.com', '9000000015', 'Male', 'India', 'Delhi', '2025-05-01', 'Regular'),
('Pooja', 'Sharma', 'pooja.sharma@example.com', '9000000016', 'Female', 'India', 'Jaipur', '2025-05-05', 'Regular'),
('Karan', 'Desai', 'karan.desai@example.com', '9000000017', 'Male', 'India', 'Surat', '2025-05-10', 'Premium'),
('Riya', 'Fernandes', 'riya.fernandes@example.com', '9000000018', 'Female', 'India', 'Goa', '2025-05-15', 'Regular'),
('Yash', 'Chopra', 'yash.chopra@example.com', '9000000019', 'Male', 'India', 'Noida', '2025-06-01', 'Premium'),
('Simran', 'Kaur', 'simran.kaur@example.com', '9000000020', 'Female', 'India', 'Amritsar', '2025-06-05', 'Regular')
```

LOADED DATA — CUSTOMERS (PHONE, GENDER AND COUNTRY OMITTED FROM DISPLAY)

customer_id	first_name	last_name	email	city	registration_date	customer_type
1	Aarav	Shah	aarav.shah@example.com	Mumbai	2025-01-05	Regular
2	Priya	Patel	priya.patel@example.com	Mumbai	2025-01-10	Premium
3	Rahul	Verma	rahul.verma@example.com	Delhi	2025-01-15	Regular
4	Sneha	Rao	sneha.rao@example.com	Bengaluru	2025-02-01	Premium
5	Kabir	Mehta	kabir.mehta@example.com	Pune	2025-02-10	Regular
6	Ananya	Das	ananya.das@example.com	Kolkata	2025-02-15	Regular
7	Vikram	Singh	vikram.singh@example.com	Jaipur	2025-03-01	Premium
8	Isha	Kapoor	isha.kapoor@example.com	Chandigarh	2025-03-05	Regular
9	Rohan	Nair	rohan.nair@example.com	Kochi	2025-03-10	Premium
10	Meera	Joshi	meera.joshi@example.com	Nashik	2025-03-15	Regular
11	Aditya	Gupta	aditya.gupta@example.com	Lucknow	2025-04-01	Regular
12	Kavya	Iyer	kavya.iyer@example.com	Chennai	2025-04-05	Premium
13	Arjun	Malhotra	arjun.malhotra@example.com	Mumbai	2025-04-10	Regular
14	Neha	Kulkarni	neha.kulkarni@example.com	Pune	2025-04-15	Premium
15	Dev	Khanna	dev.khanna@example.com	Delhi	2025-05-01	Regular
16	Pooja	Sharma	pooja.sharma@example.com	Jaipur	2025-05-05	Regular
17	Karan	Desai	karan.desai@example.com	Surat	2025-05-10	Premium
18	Riya	Fernandes	riya.fernandes@example.com	Goa	2025-05-15	Regular
19	Yash	Chopra	yash.chopra@example.com	Noida	2025-06-01	Premium
20	Simran	Kaur	simran.kaur@example.com	Amritsar	2025-06-05	Regular

SECTION

05 Operational Data Load

Steps 24–35: rooms, staff, the booking ledger and everything that hangs off it, followed by the post-load integrity checks.

The dataset is built so every query returns real output: each booking has exactly one room, one payment record and one hotel; every cancelled booking has a cancellation row; reviews and service usage exist only for stays that took place.

Step 24 — rooms (25 rows)

Five rooms per hotel — one of each room type — numbered 101, 102, 201, 202 and 301 across three floors, all `Available`.

STEP 24 — INSERT INTO ROOMS

```
INSERT INTO rooms
(hotel_id, room_type_id, room_number, floor_number, room_status)
VALUES
  (1, 1, '101', 1, 'Available'),
  (1, 2, '102', 1, 'Available'),
  (1, 3, '201', 2, 'Available'),
  (1, 4, '202', 2, 'Available'),
  (1, 5, '301', 3, 'Available'),
  (2, 1, '101', 1, 'Available'),
  (2, 2, '102', 1, 'Available'),
  (2, 3, '201', 2, 'Available'),
  (2, 4, '202', 2, 'Available'),
  (2, 5, '301', 3, 'Available'),
  (3, 1, '101', 1, 'Available'),
  (3, 2, '102', 1, 'Available'),
  (3, 3, '201', 2, 'Available'),
  (3, 4, '202', 2, 'Available'),
  (3, 5, '301', 3, 'Available'),
  (4, 1, '101', 1, 'Available'),
  (4, 2, '102', 1, 'Available'),
  (4, 3, '201', 2, 'Available'),
  (4, 4, '202', 2, 'Available'),
  (4, 5, '301', 3, 'Available'),
  (5, 1, '101', 1, 'Available'),
  (5, 2, '102', 1, 'Available'),
  (5, 3, '201', 2, 'Available'),
  (5, 4, '202', 2, 'Available'),
  (5, 5, '301', 3, 'Available')
```

LOADED DATA — ROOMS (HOTEL AND TYPE NAMES RESOLVED)

room_id	hotel_name	room_type_name	room_number	floor_number	room_status
1	Grand Palace Hotel	Standard	101	1	Available
2	Grand Palace Hotel	Deluxe	102	1	Available
3	Grand Palace Hotel	Suite	201	2	Available
4	Grand Palace Hotel	Family Room	202	2	Available
5	Grand Palace Hotel	Executive	301	3	Available
6	Sea View Resort	Standard	101	1	Available
7	Sea View Resort	Deluxe	102	1	Available
8	Sea View Resort	Suite	201	2	Available
9	Sea View Resort	Family Room	202	2	Available
10	Sea View Resort	Executive	301	3	Available
11	Royal Comfort Inn	Standard	101	1	Available
12	Royal Comfort Inn	Deluxe	102	1	Available
13	Royal Comfort Inn	Suite	201	2	Available
14	Royal Comfort Inn	Family Room	202	2	Available
15	Royal Comfort Inn	Executive	301	3	Available
16	Mountain Escape	Standard	101	1	Available
17	Mountain Escape	Deluxe	102	1	Available
18	Mountain Escape	Suite	201	2	Available
19	Mountain Escape	Family Room	202	2	Available
20	Mountain Escape	Executive	301	3	Available
21	City Business Hotel	Standard	101	1	Available
22	City Business Hotel	Deluxe	102	1	Available
23	City Business Hotel	Suite	201	2	Available
24	City Business Hotel	Family Room	202	2	Available
25	City Business Hotel	Executive	301	3	Available

Step 25 — employees (15 rows)

Three staff per hotel — manager, receptionist and a chef or housekeeper — with salaries from ₹22,000 to ₹75,000 and hire dates between 2020 and 2024.

STEP 25 — INSERT INTO EMPLOYEES

```
INSERT INTO employees
(hotel_id, employee_name, job_title, department, salary, hire_date, employment_status)
VALUES
  (1, 'Amit Sharma', 'Manager', 'Management', 75000, '2022-01-10', 'Active'),
  (1, 'Neel Patel', 'Receptionist', 'Front Office', 30000, '2023-03-15', 'Active'),
  (1, 'Sara Khan', 'Chef', 'Food', 40000, '2023-06-01', 'Active'),
  (2, 'Ramesh Naik', 'Manager', 'Management', 65000, '2022-05-12', 'Active'),
  (2, 'Anjali Rao', 'Receptionist', 'Front Office', 28000, '2023-07-20', 'Active'),
  (2, 'John Dsouza', 'Chef', 'Food', 38000, '2024-01-05', 'Active'),
  (3, 'Vivek Gupta', 'Manager', 'Management', 60000, '2021-08-10', 'Active'),
  (3, 'Pallavi Das', 'Receptionist', 'Front Office', 27000, '2024-02-15', 'Active'),
  (3, 'Manoj Kumar', 'Housekeeping Staff', 'Housekeeping', 22000, '2024-05-01', 'Active'),
  (4, 'Suresh Thakur', 'Manager', 'Management', 62000, '2022-11-10', 'Active'),
  (4, 'Nisha Verma', 'Receptionist', 'Front Office', 29000, '2023-09-15', 'Active'),
  (4, 'Deepak Joshi', 'Chef', 'Food', 36000, '2024-04-01', 'Active'),
  (5, 'Kiran Shetty', 'Manager', 'Management', 68000, '2020-12-10', 'Active'),
  (5, 'Maya Iyer', 'Receptionist', 'Front Office', 31000, '2022-08-15', 'Active'),
  (5, 'Rohit Singh', 'Housekeeping Staff', 'Housekeeping', 24000, '2024-06-01', 'Active')
```

LOADED DATA — EMPLOYEES

employee_id	hotel_id	employee_name	job_title	department	salary	hire_date	employment_status
1	1	Amit Sharma	Manager	Management	75000	2022-01-10	Active
2	1	Neel Patel	Receptionist	Front Office	30000	2023-03-15	Active
3	1	Sara Khan	Chef	Food	40000	2023-06-01	Active
4	2	Ramesh Naik	Manager	Management	65000	2022-05-12	Active
5	2	Anjali Rao	Receptionist	Front Office	28000	2023-07-20	Active
6	2	John Dsouza	Chef	Food	38000	2024-01-05	Active
7	3	Vivek Gupta	Manager	Management	60000	2021-08-10	Active
8	3	Pallavi Das	Receptionist	Front Office	27000	2024-02-15	Active
9	3	Manoj Kumar	Housekeeping Staff	Housekeeping	22000	2024-05-01	Active
10	4	Suresh Thakur	Manager	Management	62000	2022-11-10	Active
11	4	Nisha Verma	Receptionist	Front Office	29000	2023-09-15	Active
12	4	Deepak Joshi	Chef	Food	36000	2024-04-01	Active
13	5	Kiran Shetty	Manager	Management	68000	2020-12-10	Active
14	5	Maya Iyer	Receptionist	Front Office	31000	2022-08-15	Active
15	5	Rohit Singh	Housekeeping Staff	Housekeeping	24000	2024-06-01	Active

Step 26 — bookings (30 rows)

The booking ledger runs from January to May 2025 with exactly six bookings a month. The channel foreign key is added first with `ALTER TABLE`, then the rows are loaded: 25 completed and 5 cancelled, with gross values from ₹5,000 to ₹24,000 and discounts up to ₹2,000.

STEP 26A — ALTER TABLE BOOKINGS (LINK BOOKING SOURCES)

```
ALTER TABLE bookings
ADD CONSTRAINT fk_bookings_source
FOREIGN KEY (booking_source_id)
REFERENCES booking_sources(booking_source_id);
```

STEP 26B — INSERT INTO BOOKINGS

```
INSERT INTO bookings
(customer_id, hotel_id, booking_source_id, booking_date, check_in_date, check_out_date, number_of_guests,
 booking_status, total_amount, discount_amount)
VALUES
  (1,1,1,'2025-01-05','2025-01-10','2025-01-12',2,'Completed',8000,500),
  (2,1,3,'2025-01-08','2025-01-15','2025-01-18',2,'Completed',15000,1000),
  (3,2,2,'2025-01-10','2025-01-20','2025-01-23',3,'Completed',12000,500),
  (4,5,4,'2025-01-15','2025-01-25','2025-01-27',2,'Completed',11000,0),
  (5,3,1,'2025-01-20','2025-02-01','2025-02-03',2,'Completed',5000,300),
  (6,2,3,'2025-01-25','2025-02-05','2025-02-09',4,'Cancelled',18000,1000),
  (7,4,2,'2025-02-01','2025-02-10','2025-02-13',2,'Completed',12000,500),
  (8,1,5,'2025-02-05','2025-02-15','2025-02-17',2,'Completed',16000,1000),
  (9,5,1,'2025-02-10','2025-02-20','2025-02-22',2,'Completed',11000,0),
  (10,3,3,'2025-02-15','2025-02-25','2025-02-28',3,'Cancelled',9000,500),
  (11,2,4,'2025-02-20','2025-03-01','2025-03-04',2,'Completed',12000,500),
  (12,1,2,'2025-02-25','2025-03-05','2025-03-08',2,'Completed',24000,2000),
  (13,5,3,'2025-03-01','2025-03-10','2025-03-12',2,'Completed',11000,500),
  (14,4,1,'2025-03-05','2025-03-15','2025-03-18',4,'Completed',18000,1000),
  (15,3,5,'2025-03-10','2025-03-20','2025-03-22',2,'Completed',5000,0),
  (16,2,2,'2025-03-15','2025-03-25','2025-03-28',3,'Cancelled',12000,500),
  (17,1,4,'2025-03-20','2025-04-01','2025-04-04',2,'Completed',24000,1500),
  (18,2,3,'2025-03-25','2025-04-05','2025-04-08',2,'Completed',12000,500),
  (19,5,1,'2025-04-01','2025-04-10','2025-04-12',2,'Completed',11000,0),
  (20,4,5,'2025-04-05','2025-04-15','2025-04-19',5,'Completed',24000,1000),
  (1,3,2,'2025-04-10','2025-04-20','2025-04-22',2,'Completed',5000,200),
  (2,2,3,'2025-04-15','2025-04-25','2025-04-28',3,'Cancelled',12000,500),
  (3,1,1,'2025-04-20','2025-05-01','2025-05-04',2,'Completed',24000,1000),
  (4,5,4,'2025-04-25','2025-05-05','2025-05-07',2,'Completed',11000,500),
  (5,4,2,'2025-05-01','2025-05-10','2025-05-12',2,'Completed',8000,0),
  (6,1,3,'2025-05-05','2025-05-15','2025-05-18',2,'Cancelled',24000,2000),
  (7,2,1,'2025-05-10','2025-05-20','2025-05-23',3,'Completed',12000,500),
  (8,5,5,'2025-05-15','2025-05-25','2025-05-27',2,'Completed',11000,0),
  (9,3,4,'2025-05-20','2025-06-01','2025-06-04',2,'Completed',7500,500),
  (10,4,2,'2025-05-25','2025-06-05','2025-06-08',4,'Completed',18000,1000)
```

STEP 26C — VERIFICATION

```
SELECT COUNT(*) AS total_bookings
FROM bookings;

SELECT booking_status, COUNT(*) AS total
FROM bookings
GROUP BY booking_status;

SELECT *
FROM bookings
```

```
ORDER BY booking_id;
```

OUTPUT — TOTAL BOOKINGS

total_bookings
30

OUTPUT — BOOKINGS BY STATUS

booking_status	total
Cancelled	5
Completed	25

OUTPUT — 30 ROWS (BOOKINGS)

booking_id	customer_id	hotel_id	booking_source_id	booking_date	check_in_date	check_out_date	number_of_guests	booking_status	total_amount	discount_amount
1	1	1	1	2025-01-05	2025-01-10	2025-01-12	2	Completed	8000	500
2	2	1	3	2025-01-08	2025-01-15	2025-01-18	2	Completed	15000	1000
3	3	2	2	2025-01-10	2025-01-20	2025-01-23	3	Completed	12000	500
4	4	5	4	2025-01-15	2025-01-25	2025-01-27	2	Completed	11000	0
5	5	3	1	2025-01-20	2025-02-01	2025-02-03	2	Completed	5000	300
6	6	2	3	2025-01-25	2025-02-05	2025-02-09	4	Cancelled	18000	1000
7	7	4	2	2025-02-01	2025-02-10	2025-02-13	2	Completed	12000	500
8	8	1	5	2025-02-05	2025-02-15	2025-02-17	2	Completed	16000	1000
9	9	5	1	2025-02-10	2025-02-20	2025-02-22	2	Completed	11000	0
10	10	3	3	2025-02-15	2025-02-25	2025-02-28	3	Cancelled	9000	500
11	11	2	4	2025-02-20	2025-03-01	2025-03-04	2	Completed	12000	500
12	12	1	2	2025-02-25	2025-03-05	2025-03-08	2	Completed	24000	2000
13	13	5	3	2025-03-01	2025-03-10	2025-03-12	2	Completed	11000	500
14	14	4	1	2025-03-05	2025-03-15	2025-03-18	4	Completed	18000	1000
15	15	3	5	2025-03-10	2025-03-20	2025-03-22	2	Completed	5000	0
16	16	2	2	2025-03-15	2025-03-25	2025-03-28	3	Cancelled	12000	500
17	17	1	4	2025-03-20	2025-04-01	2025-04-04	2	Completed	24000	1500
18	18	2	3	2025-03-25	2025-04-05	2025-04-08	2	Completed	12000	500
19	19	5	1	2025-04-01	2025-04-10	2025-04-12	2	Completed	11000	0
20	20	4	5	2025-04-05	2025-04-15	2025-04-19	5	Completed	24000	1000
21	1	3	2	2025-04-10	2025-04-20	2025-04-22	2	Completed	5000	200
22	2	2	3	2025-04-15	2025-04-25	2025-04-28	3	Cancelled	12000	500
23	3	1	1	2025-04-20	2025-05-01	2025-05-04	2	Completed	24000	1000
24	4	5	4	2025-04-25	2025-05-05	2025-05-07	2	Completed	11000	500
25	5	4	2	2025-05-01	2025-05-10	2025-05-12	2	Completed	8000	0
26	6	1	3	2025-05-05	2025-05-15	2025-05-18	2	Cancelled	24000	2000
27	7	2	1	2025-05-10	2025-05-20	2025-05-23	3	Completed	12000	500
28	8	5	5	2025-05-15	2025-05-25	2025-05-27	2	Completed	11000	0
29	9	3	4	2025-05-20	2025-06-01	2025-06-04	2	Completed	7500	500
30	10	4	2	2025-05-25	2025-06-05	2025-06-08	4	Completed	18000	1000

Step 27 — booking_rooms (30 rows)

One room per booking with the nightly rate charged. Rates match the room-type list price in most cases and diverge where the hotel priced dynamically (for example a Deluxe room sold at ₹5,000 against a ₹4,000 list price).

STEP 27 — INSERT INTO BOOKING ROOMS

```
INSERT INTO booking_rooms
(booking_id, room_id, nightly_rate)
VALUES
  (1,1,4000),
  (2,2,5000),
  (3,6,4000),
  (4,22,5500),
  (5,11,2500),
  (6,7,4500),
  (7,16,4000),
  (8,3,8000),
  (9,23,5500),
  (10,12,3000),
  (11,8,4000),
  (12,4,8000),
  (13,24,5500),
  (14,19,6000),
  (15,13,2500),
  (16,9,4000),
  (17,5,8000),
  (18,10,4000),
  (19,25,5500),
  (20,20,6000),
  (21,14,2500),
  (22,6,4000),
  (23,3,8000),
  (24,21,5500),
  (25,17,4000),
  (26,4,8000),
  (27,7,4000),
  (28,22,5500),
  (29,15,2500),
  (30,18,4000)
```

LOADED DATA — BOOKING_ROOMS

booking_room_id	booking_id	room_id	nightly_rate
1	1	1	4000
2	2	2	5000
3	3	6	4000
4	4	22	5500
5	5	11	2500
6	6	7	4500
7	7	16	4000
8	8	3	8000
9	9	23	5500
10	10	12	3000
11	11	8	4000
12	12	4	8000
13	13	24	5500
14	14	19	6000
15	15	13	2500
16	16	9	4000
17	17	5	8000
18	18	10	4000
19	19	25	5500
20	20	20	6000
21	21	14	2500
22	22	6	4000
23	23	3	8000
24	24	21	5500
25	25	17	4000
26	26	4	8000
27	27	7	4000
28	28	22	5500
29	29	15	2500
30	30	18	4000

Step 28 — payments (30 rows)

Payments by credit card, debit card, UPI and cash. Paid amounts equal the net booking value (gross less discount); cancelled bookings carry a zero `Refunded` record, and two bookings are `Pending`.

STEP 28 — INSERT INTO PAYMENTS

```
INSERT INTO payments
(booking_id, payment_date, payment_method, payment_amount, payment_status, transaction_reference)
VALUES
  (1,'2025-01-05','Credit Card',7500,'Paid','TXN1001'),
  (2,'2025-01-08','UPI',14000,'Paid','TXN1002'),
  (3,'2025-01-10','Debit Card',11500,'Paid','TXN1003'),
  (4,'2025-01-15','Credit Card',11000,'Paid','TXN1004'),
  (5,'2025-01-20','UPI',4700,'Paid','TXN1005'),
  (6,'2025-01-25','UPI',0,'Refunded','TXN1006'),
  (7,'2025-02-01','Credit Card',11500,'Paid','TXN1007'),
  (8,'2025-02-05','Debit Card',15000,'Paid','TXN1008'),
  (9,'2025-02-10','UPI',11000,'Paid','TXN1009'),
  (10,'2025-02-15','UPI',0,'Refunded','TXN1010'),
  (11,'2025-02-20','Credit Card',11500,'Paid','TXN1011'),
  (12,'2025-02-25','Credit Card',22000,'Paid','TXN1012'),
  (13,'2025-03-01','UPI',10500,'Paid','TXN1013'),
  (14,'2025-03-05','Debit Card',17000,'Paid','TXN1014'),
  (15,'2025-03-10','Cash',5000,'Paid','TXN1015'),
  (16,'2025-03-15','UPI',0,'Refunded','TXN1016'),
  (17,'2025-03-20','Credit Card',22500,'Paid','TXN1017'),
  (18,'2025-03-25','UPI',11500,'Paid','TXN1018'),
  (19,'2025-04-01','Debit Card',11000,'Paid','TXN1019'),
  (20,'2025-04-05','Credit Card',23000,'Paid','TXN1020'),
  (21,'2025-04-10','UPI',4800,'Paid','TXN1021'),
  (22,'2025-04-15','UPI',0,'Refunded','TXN1022'),
  (23,'2025-04-20','Credit Card',23000,'Paid','TXN1023'),
  (24,'2025-04-25','Debit Card',10500,'Paid','TXN1024'),
  (25,'2025-05-01','UPI',8000,'Paid','TXN1025'),
  (26,'2025-05-05','Credit Card',22000,'Paid','TXN1026'),
  (27,'2025-05-10','UPI',11500,'Paid','TXN1027'),
  (28,'2025-05-15','Debit Card',11000,'Paid','TXN1028'),
  (29,'2025-05-20','Cash',7000,'Pending','TXN1029'),
  (30,'2025-05-25','Credit Card',18000,'Pending','TXN1030')
```

STEP 28B — VERIFICATION

```
SELECT payment_status, COUNT(*) AS payment_count,
  SUM(payment_amount) AS total_payment
FROM payments
GROUP BY payment_status;

SELECT
  b.booking_id,
  h.hotel_name,
  p.payment_amount,
  p.payment_status
FROM bookings b
JOIN hotels h
  ON b.hotel_id = h.hotel_id
JOIN payments p
  ON b.booking_id = p.booking_id
ORDER BY b.booking_id;
```

OUTPUT — PAYMENTS BY STATUS

payment_status	payment_count	total_payment
Paid	24	311000
Pending	2	25000
Refunded	4	0

OUTPUT — 30 ROWS (PAYMENT PER BOOKING)

booking_id	hotel_name	payment_amount	payment_status
1	Grand Palace Hotel	7500	Paid
2	Grand Palace Hotel	14000	Paid
3	Sea View Resort	11500	Paid
4	City Business Hotel	11000	Paid
5	Royal Comfort Inn	4700	Paid
6	Sea View Resort	0	Refunded
7	Mountain Escape	11500	Paid
8	Grand Palace Hotel	15000	Paid
9	City Business Hotel	11000	Paid
10	Royal Comfort Inn	0	Refunded
11	Sea View Resort	11500	Paid
12	Grand Palace Hotel	22000	Paid
13	City Business Hotel	10500	Paid
14	Mountain Escape	17000	Paid
15	Royal Comfort Inn	5000	Paid
16	Sea View Resort	0	Refunded
17	Grand Palace Hotel	22500	Paid
18	Sea View Resort	11500	Paid
19	City Business Hotel	11000	Paid
20	Mountain Escape	23000	Paid
21	Royal Comfort Inn	4800	Paid
22	Sea View Resort	0	Refunded
23	Grand Palace Hotel	23000	Paid
24	City Business Hotel	10500	Paid
25	Mountain Escape	8000	Paid
26	Grand Palace Hotel	22000	Paid
27	Sea View Resort	11500	Paid
28	City Business Hotel	11000	Paid
29	Royal Comfort Inn	7000	Pending
30	Mountain Escape	18000	Pending

Step 29 — cancellations (5 rows)

STEP 29 — INSERT INTO CANCELLATIONS

```
INSERT INTO cancellations
(booking_id, cancellation_date, cancellation_reason, refund_amount)
VALUES
  (6, '2025-01-28', 'Change in travel plan', 18000),
  (10, '2025-02-20', 'Booking made by mistake', 9000),
  (16, '2025-03-18', 'Unexpected emergency', 12000),
  (22, '2025-04-18', 'Date change required', 12000),
  (26, '2025-05-10', 'Personal reasons', 24000)
```

LOADED DATA — CANCELLATIONS

cancellation_id	booking_id	cancellation_date	cancellation_reason	refund_amount
1	6	2025-01-28	Change in travel plan	18000
2	10	2025-02-20	Booking made by mistake	9000
3	16	2025-03-18	Unexpected emergency	12000
4	22	2025-04-18	Date change required	12000
5	26	2025-05-10	Personal reasons	24000

Step 30 — reviews (20 rows)

Twenty reviews from 3.0 to 5.0 stars for completed stays; cancelled bookings and the two June check-ins have none.

STEP 30 — INSERT INTO REVIEWS

```sql
INSERT INTO reviews
(booking_id, customer_id, rating, review_date, review_text)
VALUES
  (1, 1, 4.5, '2025-01-12', 'Excellent service and clean room'),
  (2, 2, 5.0, '2025-01-18', 'Amazing luxury experience'),
  (3, 3, 4.0, '2025-01-23', 'Comfortable stay'),
  (4, 4, 3.5, '2025-01-27', 'Good location but slow service'),
  (5, 5, 4.0, '2025-02-03', 'Value for money'),
  (7, 7, 4.5, '2025-02-13', 'Beautiful resort'),
  (8, 8, 5.0, '2025-02-17', 'Outstanding hospitality'),
  (9, 9, 4.0, '2025-02-22', 'Nice business hotel'),
  (11, 11, 3.0, '2025-03-04', 'Average room service'),
  (12, 12, 4.5, '2025-03-08', 'Spacious and comfortable suite'),
  (13, 13, 4.0, '2025-03-12', 'Good overall experience'),
  (14, 14, 4.5, '2025-03-18', 'Great family stay'),
  (15, 15, 3.5, '2025-03-22', 'Room was a little small'),
  (17, 17, 5.0, '2025-04-04', 'Excellent staff and facilities'),
  (18, 18, 4.0, '2025-04-08', 'Enjoyed the stay'),
  (19, 19, 4.5, '2025-04-12', 'Very clean hotel'),
  (20, 20, 5.0, '2025-04-19', 'Perfect family vacation'),
  (21, 1, 4.0, '2025-04-22', 'Good value'),
  (23, 3, 4.5, '2025-05-04', 'Would visit again'),
  (24, 4, 3.5, '2025-05-07', 'Could improve breakfast')
```

LOADED DATA — REVIEWS

review_id	booking_id	customer_id	rating	review_date	review_text
1	1	1	4.5	2025-01-12	Excellent service and clean room
2	2	2	5	2025-01-18	Amazing luxury experience
3	3	3	4	2025-01-23	Comfortable stay
4	4	4	3.5	2025-01-27	Good location but slow service
5	5	5	4	2025-02-03	Value for money
6	7	7	4.5	2025-02-13	Beautiful resort
7	8	8	5	2025-02-17	Outstanding hospitality
8	9	9	4	2025-02-22	Nice business hotel
9	11	11	3	2025-03-04	Average room service
10	12	12	4.5	2025-03-08	Spacious and comfortable suite
11	13	13	4	2025-03-12	Good overall experience
12	14	14	4.5	2025-03-18	Great family stay
13	15	15	3.5	2025-03-22	Room was a little small
14	17	17	5	2025-04-04	Excellent staff and facilities
15	18	18	4	2025-04-08	Enjoyed the stay
16	19	19	4.5	2025-04-12	Very clean hotel
17	20	20	5	2025-04-19	Perfect family vacation
18	21	1	4	2025-04-22	Good value
19	23	3	4.5	2025-05-04	Would visit again
20	24	4	3.5	2025-05-07	Could improve breakfast

Step 31 — service_usage (20 rows)

Add-on consumption during stays; `total_cost` equals list price multiplied by quantity in every row.

STEP 31 — INSERT INTO SERVICE USAGE

```sql
INSERT INTO service_usage
(booking_id, service_id, usage_date, quantity, total_cost)
VALUES
  (1, 1, '2025-01-11', 2, 1000),
  (2, 2, '2025-01-16', 1, 2000),
  (3, 1, '2025-01-21', 3, 1500),
  (4, 4, '2025-01-26', 1, 1500),
  (5, 3, '2025-02-02', 2, 600),
  (7, 1, '2025-02-11', 2, 1000),
  (8, 2, '2025-02-16', 1, 2000),
  (9, 5, '2025-02-21', 2, 1400),
  (11, 4, '2025-03-02', 1, 1500),
  (12, 1, '2025-03-06', 2, 1000),
  (13, 3, '2025-03-11', 1, 300),
  (14, 1, '2025-03-16', 4, 2000),
  (15, 5, '2025-03-21', 1, 700),
  (17, 2, '2025-04-02', 1, 2000),
  (18, 4, '2025-04-06', 1, 1500),
  (19, 1, '2025-04-11', 2, 1000),
  (20, 5, '2025-04-16', 3, 2100),
  (21, 3, '2025-04-21', 1, 300),
```

```
  (23, 2, '2025-05-02', 1, 2000),
  (24, 1, '2025-05-06', 2, 1000)
```

LOADED DATA — SERVICE_USAGE

usage_id	booking_id	service_id	usage_date	quantity	total_cost
1	1	1	2025-01-11	2	1000
2	2	2	2025-01-16	1	2000
3	3	1	2025-01-21	3	1500
4	4	4	2025-01-26	1	1500
5	5	3	2025-02-02	2	600
6	7	1	2025-02-11	2	1000
7	8	2	2025-02-16	1	2000
8	9	5	2025-02-21	2	1400
9	11	4	2025-03-02	1	1500
10	12	1	2025-03-06	2	1000
11	13	3	2025-03-11	1	300
12	14	1	2025-03-16	4	2000
13	15	5	2025-03-21	1	700
14	17	2	2025-04-02	1	2000
15	18	4	2025-04-06	1	1500
16	19	1	2025-04-11	2	1000
17	20	5	2025-04-16	3	2100
18	21	3	2025-04-21	1	300
19	23	2	2025-05-02	1	2000
20	24	1	2025-05-06	2	1000

Step 32 — expenses (20 rows)

Four monthly postings per hotel from January to April 2025 — Utilities, Maintenance, Staff and Marketing.

STEP 32 — INSERT INTO EXPENSES

```
INSERT INTO expenses
(hotel_id, expense_date, expense_category, expense_description, expense_amount)
VALUES
  (1, '2025-01-31', 'Utilities', 'Electricity and water', 25000),
  (1, '2025-02-28', 'Maintenance', 'Room maintenance', 12000),
  (1, '2025-03-31', 'Staff', 'Monthly operating payroll', 90000),
  (1, '2025-04-30', 'Marketing', 'Online promotions', 15000),
  (2, '2025-01-31', 'Utilities', 'Electricity and water', 18000),
  (2, '2025-02-28', 'Maintenance', 'Pool and room maintenance', 10000),
  (2, '2025-03-31', 'Staff', 'Monthly operating payroll', 75000),
  (2, '2025-04-30', 'Marketing', 'Travel promotions', 12000),
  (3, '2025-01-31', 'Utilities', 'Electricity and water', 14000),
  (3, '2025-02-28', 'Maintenance', 'Room maintenance', 8000),
  (3, '2025-03-31', 'Staff', 'Monthly operating payroll', 65000),
  (3, '2025-04-30', 'Marketing', 'Business travel campaign', 10000),
  (4, '2025-01-31', 'Utilities', 'Electricity and water', 16000),
  (4, '2025-02-28', 'Maintenance', 'Property maintenance', 9000),
  (4, '2025-03-31', 'Staff', 'Monthly operating payroll', 70000),
  (4, '2025-04-30', 'Marketing', 'Holiday promotions', 11000),
  (5, '2025-01-31', 'Utilities', 'Electricity and water', 20000),
  (5, '2025-02-28', 'Maintenance', 'Room maintenance', 11000),
  (5, '2025-03-31', 'Staff', 'Monthly operating payroll', 85000),
  (5, '2025-04-30', 'Marketing', 'Corporate promotions', 14000)
```

LOADED DATA — EXPENSES

expense_id	hotel_id	expense_date	expense_category	expense_description	expense_amount
1	1	2025-01-31	Utilities	Electricity and water	25000
2	1	2025-02-28	Maintenance	Room maintenance	12000
3	1	2025-03-31	Staff	Monthly operating payroll	90000
4	1	2025-04-30	Marketing	Online promotions	15000
5	2	2025-01-31	Utilities	Electricity and water	18000
6	2	2025-02-28	Maintenance	Pool and room maintenance	10000
7	2	2025-03-31	Staff	Monthly operating payroll	75000
8	2	2025-04-30	Marketing	Travel promotions	12000
9	3	2025-01-31	Utilities	Electricity and water	14000
10	3	2025-02-28	Maintenance	Room maintenance	8000
11	3	2025-03-31	Staff	Monthly operating payroll	65000
12	3	2025-04-30	Marketing	Business travel campaign	10000
13	4	2025-01-31	Utilities	Electricity and water	16000
14	4	2025-02-28	Maintenance	Property maintenance	9000
15	4	2025-03-31	Staff	Monthly operating payroll	70000
16	4	2025-04-30	Marketing	Holiday promotions	11000
17	5	2025-01-31	Utilities	Electricity and water	20000
18	5	2025-02-28	Maintenance	Room maintenance	11000
19	5	2025-03-31	Staff	Monthly operating payroll	85000
20	5	2025-04-30	Marketing	Corporate promotions	14000

Steps 33–35 — Load verification and integrity checks

Row counts are checked first — the simplest guard against a partially applied script — followed by two integrity checks that should return an empty set.

STEP 33 — ROW COUNTS OF EVERY LOADED TABLE

```sql
SELECT 'customers' AS table_name, COUNT(*) AS total_rows FROM customers
UNION ALL
SELECT 'hotels', COUNT(*) FROM hotels
UNION ALL
SELECT 'rooms', COUNT(*) FROM rooms
UNION ALL
SELECT 'bookings', COUNT(*) FROM bookings
UNION ALL
SELECT 'booking_rooms', COUNT(*) FROM booking_rooms
UNION ALL
SELECT 'payments', COUNT(*) FROM payments
UNION ALL
SELECT 'cancellations', COUNT(*) FROM cancellations
UNION ALL
SELECT 'reviews', COUNT(*) FROM reviews
UNION ALL
SELECT 'service_usage', COUNT(*) FROM service_usage
UNION ALL
SELECT 'employees', COUNT(*) FROM employees
UNION ALL
SELECT 'expenses', COUNT(*) FROM expenses;
```

OUTPUT — 11 ROWS

table_name	total_rows
customers	20
hotels	5
rooms	25
bookings	30
booking_rooms	30
payments	30
cancellations	5
reviews	20
service_usage	20
employees	15
expenses	20

STEP 34 — INVALID DATE RANGES

```sql
SELECT booking_id, check_in_date, check_out_date
FROM bookings
WHERE check_out_date <= check_in_date;
```

RESULT: EMPTY SET

No booking checks out on or before its check-in date, which is the correct and desired outcome.

STEP 35 — BOOKINGS WITHOUT A MATCHING HOTEL

```sql
SELECT b.booking_id, b.hotel_id
FROM bookings b
LEFT JOIN hotels h
  ON b.hotel_id = h.hotel_id
WHERE h.hotel_id IS NULL;
```

RESULT: EMPTY SET

The `LEFT JOIN ... WHERE h.hotel_id IS NULL` anti-join finds no orphan bookings. The same pattern is a useful template for checking any foreign key.

SECTION

06 Hotel Performance Analytics

Questions 1–5: the group-wide KPI report, profitability and risk, revenue trend, booking-channel economics and guest value.

COMPLETED REVENUE	CANCELLATION RATE	AVG GUEST RATING	ADD-ON SERVICES	OPERATING EXPENSES
₹327,500	16.67%	4.22	₹26,400	₹590,000
25 of 30 bookings	5 bookings, ₹75,000 refunded	20 reviews, none below 3.0	20 usage records	Jan–Apr postings

Each question below is a single statement built from named CTE steps, so the logic reads top to bottom like a report specification. Every result set is shown in full.

HOW THE OUTPUTS WERE PRODUCED

MySQL 8 was not available in the authoring environment, so every statement was executed on SQLite 3.45 with equivalent implementations of `DATEDIFF`, `DATE_FORMAT`, `CONCAT` and `GROUP_CONCAT`. Averages are shown rounded to two decimals. Re-run the script on MySQL 8 to confirm the figures on your own server.

Question 1 — Hotel revenue and performance report

The largest query in the project: twenty-one CTE sections compute booking, revenue, payment, cancellation, review, service, expense, channel, stay, lead-time, room, monthly-growth, retention and discount metrics for each hotel over 1 January – 31 May 2025. They are combined in `hotel_metrics`, converted to ratios in `hotel_kpis`, and labelled with a `recommended_action` by a `CASE` cascade in the final select. A one-row `report_period` CTE is `CROSS JOIN`ed into every step so the window is defined in exactly one place.

QUESTION 1 — HOTEL REVENUE & PERFORMANCE REPORT

```
WITH
-- SECTION 1: REPORTING PERIOD
report_period AS (
  SELECT
    DATE('2025-01-01') AS start_date,
    DATE('2025-05-31') AS end_date
),
-- SECTION 2: HOTEL MASTER INFORMATION
hotel_base AS (
  SELECT
    h.hotel_id,
    h.hotel_name,
    h.city,
    h.country,
    h.star_rating,
    h.hotel_type,
    h.total_rooms
  FROM hotels h
),
-- SECTION 3: OVERALL BOOKING PERFORMANCE
booking_summary AS (
  SELECT
    b.hotel_id,
    COUNT(*) AS total_bookings,
    SUM(
    CASE
      WHEN b.booking_status = 'Completed'
        THEN 1 ELSE 0
    END
  ) AS completed_bookings,
    SUM(
    CASE
      WHEN b.booking_status = 'Cancelled'
        THEN 1 ELSE 0
    END
  ) AS cancelled_bookings,
    SUM(
    CASE
      WHEN b.booking_status = 'Pending'
        THEN 1 ELSE 0
    END
  ) AS pending_bookings,
    SUM(
    CASE
```

```
      WHEN b.booking_status = 'Confirmed'
        THEN 1 ELSE 0
    END
  ) AS confirmed_bookings,
    COUNT(DISTINCT b.customer_id) AS unique_customers,
    SUM(b.number_of_guests) AS total_guests
  FROM bookings b
  CROSS JOIN report_period rp
  WHERE b.booking_date BETWEEN rp.start_date AND rp.end_date
  GROUP BY b.hotel_id
),
-- SECTION 4: COMPLETED BOOKING REVENUE
completed_revenue AS (
  SELECT
    b.hotel_id,
    SUM(b.total_amount) AS gross_booking_value,
    SUM(b.discount_amount) AS total_discounts,
    SUM(b.total_amount - b.discount_amount)
    AS net_booking_revenue,
    AVG(b.total_amount - b.discount_amount)
    AS average_booking_value,
    AVG(b.discount_amount) AS average_discount,
    MAX(b.total_amount - b.discount_amount)
    AS highest_booking_value,
    MIN(b.total_amount - b.discount_amount)
    AS lowest_booking_value
  FROM bookings b
  CROSS JOIN report_period rp
  WHERE b.booking_status = 'Completed'
    AND b.booking_date BETWEEN rp.start_date AND rp.end_date
  GROUP BY b.hotel_id
),
-- SECTION 5: PAYMENT COLLECTION PERFORMANCE
payment_summary AS (
  SELECT
    b.hotel_id,
    COUNT(p.payment_id) AS payment_records,
    SUM(
    CASE
      WHEN p.payment_status = 'Paid'
        THEN p.payment_amount ELSE 0
    END
  ) AS successful_collections,
    SUM(
    CASE
      WHEN p.payment_status = 'Pending'
        THEN p.payment_amount ELSE 0
    END
  ) AS pending_payment_amount,
    SUM(
    CASE
      WHEN p.payment_status = 'Refunded'
        THEN p.payment_amount ELSE 0
    END
  ) AS refunded_payment_records_amount,
    SUM(
    CASE
      WHEN p.payment_status = 'Failed'
        THEN 1 ELSE 0
    END
  ) AS failed_payments,
    COUNT(
    DISTINCT CASE
      WHEN p.payment_status = 'Paid'
        THEN p.payment_method
    END
  ) AS payment_methods_used
  FROM bookings b
  JOIN payments p
    ON b.booking_id = p.booking_id
  CROSS JOIN report_period rp
  WHERE b.booking_date BETWEEN rp.start_date AND rp.end_date
  GROUP BY b.hotel_id
),
-- SECTION 6: CANCELLATION ANALYSIS
cancellation_summary AS (
  SELECT
    b.hotel_id,
    COUNT(c.cancellation_id) AS cancellation_records,
    SUM(COALESCE(c.refund_amount, 0))
    AS total_refund_amount,
    AVG(COALESCE(c.refund_amount, 0))
    AS average_refund_amount
  FROM bookings b
  JOIN cancellations c
    ON b.booking_id = c.booking_id
  CROSS JOIN report_period rp
  WHERE b.booking_date BETWEEN rp.start_date AND rp.end_date
  GROUP BY b.hotel_id
),
```

```sql
-- SECTION 7: CUSTOMER REVIEW ANALYSIS
review_summary AS (
  SELECT
    b.hotel_id,
    COUNT(r.review_id) AS total_reviews,
    ROUND(AVG(r.rating), 2) AS average_rating,
    SUM(
    CASE WHEN r.rating >= 4 THEN 1 ELSE 0 END
  ) AS positive_reviews,
    SUM(
    CASE WHEN r.rating < 3 THEN 1 ELSE 0 END
  ) AS low_rating_reviews,
    MAX(r.rating) AS highest_rating,
    MIN(r.rating) AS lowest_rating
  FROM bookings b
  JOIN reviews r
    ON b.booking_id = r.booking_id
  CROSS JOIN report_period rp
  WHERE b.booking_date BETWEEN rp.start_date AND rp.end_date
  GROUP BY b.hotel_id
),
-- SECTION 8: ADDITIONAL SERVICE REVENUE
service_summary AS (
  SELECT
    b.hotel_id,
    COUNT(su.usage_id) AS service_transactions,
    SUM(COALESCE(su.quantity, 0)) AS services_quantity,
    SUM(COALESCE(su.total_cost, 0))
    AS additional_service_revenue,
    AVG(su.total_cost) AS average_service_transaction
  FROM bookings b
  JOIN service_usage su
    ON b.booking_id = su.booking_id
  CROSS JOIN report_period rp
  WHERE b.booking_date BETWEEN rp.start_date AND rp.end_date
    AND b.booking_status = 'Completed'
  GROUP BY b.hotel_id
),
-- SECTION 9: OPERATING EXPENSE ANALYSIS
expense_summary AS (
  SELECT
    e.hotel_id,
    SUM(e.expense_amount) AS total_operating_expenses,
    AVG(e.expense_amount) AS average_expense,
    COUNT(*) AS expense_records,
    SUM(
    CASE
      WHEN e.expense_category = 'Utilities'
        THEN e.expense_amount ELSE 0
    END
  ) AS utility_expenses,
    SUM(
    CASE
      WHEN e.expense_category = 'Maintenance'
        THEN e.expense_amount ELSE 0
    END
  ) AS maintenance_expenses,
    SUM(
    CASE
      WHEN e.expense_category = 'Staff'
        THEN e.expense_amount ELSE 0
    END
  ) AS staff_expenses,
    SUM(
    CASE
      WHEN e.expense_category = 'Marketing'
        THEN e.expense_amount ELSE 0
    END
  ) AS marketing_expenses
  FROM expenses e
  CROSS JOIN report_period rp
  WHERE e.expense_date BETWEEN rp.start_date AND rp.end_date
  GROUP BY e.hotel_id
),
-- SECTION 10: BOOKING SOURCE ANALYSIS
source_summary AS (
  SELECT
    b.hotel_id,
    COUNT(*) AS sourced_bookings,
    COUNT(DISTINCT bs.source_name) AS sources_used,
    SUM(
    CASE
      WHEN bs.source_category = 'Direct'
        THEN 1 ELSE 0
    END
  ) AS direct_bookings,
    SUM(
    CASE
      WHEN bs.source_category = 'OTA'
        THEN 1 ELSE 0
```

```
    END
  ) AS ota_bookings,
    AVG(bs.commission_percentage)
    AS average_source_commission_pct
  FROM bookings b
  JOIN booking_sources bs
    ON b.booking_source_id = bs.booking_source_id
  CROSS JOIN report_period rp
  WHERE b.booking_date BETWEEN rp.start_date AND rp.end_date
  GROUP BY b.hotel_id
),
-- SECTION 11: LENGTH OF STAY ANALYSIS
stay_summary AS (
  SELECT
    b.hotel_id,
    SUM(
    DATEDIFF(b.check_out_date, b.check_in_date)
  ) AS completed_room_nights_booked,
    AVG(
    DATEDIFF(b.check_out_date, b.check_in_date)
  ) AS average_length_of_stay,
    MAX(
    DATEDIFF(b.check_out_date, b.check_in_date)
  ) AS longest_stay,
    MIN(
    DATEDIFF(b.check_out_date, b.check_in_date)
  ) AS shortest_stay
  FROM bookings b
  CROSS JOIN report_period rp
  WHERE b.booking_status = 'Completed'
    AND b.booking_date BETWEEN rp.start_date AND rp.end_date
    AND b.check_out_date > b.check_in_date
  GROUP BY b.hotel_id
),
-- SECTION 12: BOOKING LEAD TIME
lead_time_summary AS (
  SELECT
    b.hotel_id,
    AVG(
    DATEDIFF(b.check_in_date, b.booking_date)
  ) AS average_lead_time_days,
    MAX(
    DATEDIFF(b.check_in_date, b.booking_date)
  ) AS maximum_lead_time_days,
    MIN(
    DATEDIFF(b.check_in_date, b.booking_date)
  ) AS minimum_lead_time_days
  FROM bookings b
  CROSS JOIN report_period rp
  WHERE b.booking_date BETWEEN rp.start_date AND rp.end_date
    AND b.check_in_date >= b.booking_date
  GROUP BY b.hotel_id
),
-- SECTION 13: ROOM TYPE PERFORMANCE
room_type_summary AS (
  SELECT
    b.hotel_id,
    COUNT(br.booking_room_id) AS room_assignments,
    COUNT(DISTINCT rt.room_type_id) AS room_types_booked,
    SUM(
    COALESCE(br.nightly_rate, 0)
    * DATEDIFF(b.check_out_date, b.check_in_date)
  ) AS estimated_room_charge,
    AVG(br.nightly_rate) AS average_nightly_rate
  FROM bookings b
  JOIN booking_rooms br
    ON b.booking_id = br.booking_id
  JOIN rooms r
    ON br.room_id = r.room_id
  JOIN room_types rt
    ON r.room_type_id = rt.room_type_id
  CROSS JOIN report_period rp
  WHERE b.booking_status = 'Completed'
    AND b.booking_date BETWEEN rp.start_date AND rp.end_date
  GROUP BY b.hotel_id
),
-- SECTION 14: MONTHLY REVENUE
monthly_revenue AS (
  SELECT
    b.hotel_id,
    DATE_FORMAT(b.booking_date, '%Y-%m') AS revenue_month,
    SUM(b.total_amount - b.discount_amount)
    AS monthly_net_revenue,
    COUNT(*) AS monthly_completed_bookings
  FROM bookings b
  CROSS JOIN report_period rp
  WHERE b.booking_status = 'Completed'
    AND b.booking_date BETWEEN rp.start_date AND rp.end_date
  GROUP BY
    b.hotel_id,
```

```
    DATE_FORMAT(b.booking_date, '%Y-%m')
),
-- SECTION 15: MONTH-OVER-MONTH REVENUE GROWTH
monthly_growth AS (
  SELECT
    hotel_id,
    revenue_month,
    monthly_net_revenue,
    monthly_completed_bookings,
    LAG(monthly_net_revenue) OVER (
    PARTITION BY hotel_id
    ORDER BY revenue_month
  ) AS previous_month_revenue,
    LAG(monthly_completed_bookings) OVER (
    PARTITION BY hotel_id
    ORDER BY revenue_month
  ) AS previous_month_bookings
  FROM monthly_revenue
),
-- SECTION 16: MOST RECENT MONTHLY PERFORMANCE
latest_month AS (
  SELECT
    mg.*,
    ROW_NUMBER() OVER (
    PARTITION BY hotel_id
    ORDER BY revenue_month DESC
  ) AS month_rank
  FROM monthly_growth mg
),
-- SECTION 17: CUSTOMER RETENTION
customer_repeat_summary AS (
  SELECT
    b.hotel_id,
    COUNT(*) AS completed_customer_bookings,
    COUNT(DISTINCT b.customer_id) AS distinct_completed_customers,
    SUM(
    CASE
      WHEN customer_booking_counts.booking_count > 1
        THEN 1 ELSE 0
    END
  ) AS repeat_customer_bookings
  FROM bookings b
  JOIN (
    SELECT
      customer_id,
      COUNT(*) AS booking_count
    FROM bookings
    WHERE booking_status = 'Completed'
    GROUP BY customer_id
  ) customer_booking_counts
    ON b.customer_id = customer_booking_counts.customer_id
  CROSS JOIN report_period rp
  WHERE b.booking_status = 'Completed'
    AND b.booking_date BETWEEN rp.start_date AND rp.end_date
  GROUP BY b.hotel_id
),
-- SECTION 18: DISCOUNT PERFORMANCE
discount_summary AS (
  SELECT
    b.hotel_id,
    COUNT(*) AS discounted_bookings,
    SUM(b.discount_amount) AS total_discount_given,
    AVG(
    CASE
      WHEN b.total_amount > 0
        THEN b.discount_amount / b.total_amount * 100
      ELSE 0
    END
  ) AS average_discount_pct
  FROM bookings b
  CROSS JOIN report_period rp
  WHERE b.booking_status = 'Completed'
    AND b.booking_date BETWEEN rp.start_date AND rp.end_date
  GROUP BY b.hotel_id
),
-- SECTION 19: COMBINE HOTEL PERFORMANCE METRICS
hotel_metrics AS (
  SELECT
    hb.hotel_id,
    hb.hotel_name,
    hb.city,
    hb.star_rating,
    hb.hotel_type,
    hb.total_rooms,
    COALESCE(bs.total_bookings, 0) AS total_bookings,
    COALESCE(bs.completed_bookings, 0) AS completed_bookings,
    COALESCE(bs.cancelled_bookings, 0) AS cancelled_bookings,
    COALESCE(bs.unique_customers, 0) AS unique_customers,
    COALESCE(bs.total_guests, 0) AS total_guests,
    COALESCE(cr.gross_booking_value, 0) AS gross_booking_value,
```

```
    COALESCE(cr.total_discounts, 0) AS total_discounts,
    COALESCE(cr.net_booking_revenue, 0) AS net_booking_revenue,
    COALESCE(cr.average_booking_value, 0) AS average_booking_value,
    COALESCE(ps.successful_collections, 0) AS successful_collections,
    COALESCE(ps.pending_payment_amount, 0) AS pending_payment_amount,
    COALESCE(ps.failed_payments, 0) AS failed_payments,
    COALESCE(cs.total_refund_amount, 0) AS total_refund_amount,
    COALESCE(rs.average_rating, 0) AS average_rating,
    COALESCE(rs.total_reviews, 0) AS total_reviews,
    COALESCE(rs.low_rating_reviews, 0) AS low_rating_reviews,
    COALESCE(ss.additional_service_revenue, 0)
    AS additional_service_revenue,
    COALESCE(es.total_operating_expenses, 0)
    AS total_operating_expenses,
    COALESCE(es.utility_expenses, 0) AS utility_expenses,
    COALESCE(es.maintenance_expenses, 0) AS maintenance_expenses,
    COALESCE(es.staff_expenses, 0) AS staff_expenses,
    COALESCE(es.marketing_expenses, 0) AS marketing_expenses,
    COALESCE(src.direct_bookings, 0) AS direct_bookings,
    COALESCE(src.ota_bookings, 0) AS ota_bookings,
    COALESCE(st.completed_room_nights_booked, 0)
    AS completed_room_nights_booked,
    COALESCE(st.average_length_of_stay, 0)
    AS average_length_of_stay,
    COALESCE(lt.average_lead_time_days, 0)
    AS average_lead_time_days,
    COALESCE(rt.estimated_room_charge, 0)
    AS estimated_room_charge,
    COALESCE(dc.average_discount_pct, 0)
    AS average_discount_pct,
    lm.revenue_month AS latest_revenue_month,
    lm.monthly_net_revenue AS latest_month_revenue,
    lm.previous_month_revenue,
    CASE
    WHEN lm.previous_month_revenue IS NULL THEN NULL
    WHEN lm.previous_month_revenue = 0 THEN NULL
    ELSE ROUND(
      (
        lm.monthly_net_revenue
        - lm.previous_month_revenue
      ) / lm.previous_month_revenue * 100,
      2
    )
  END AS latest_month_growth_pct
  FROM hotel_base hb
  LEFT JOIN booking_summary bs
    ON hb.hotel_id = bs.hotel_id
  LEFT JOIN completed_revenue cr
    ON hb.hotel_id = cr.hotel_id
  LEFT JOIN payment_summary ps
    ON hb.hotel_id = ps.hotel_id
  LEFT JOIN cancellation_summary cs
    ON hb.hotel_id = cs.hotel_id
  LEFT JOIN review_summary rs
    ON hb.hotel_id = rs.hotel_id
  LEFT JOIN service_summary ss
    ON hb.hotel_id = ss.hotel_id
  LEFT JOIN expense_summary es
    ON hb.hotel_id = es.hotel_id
  LEFT JOIN source_summary src
    ON hb.hotel_id = src.hotel_id
  LEFT JOIN stay_summary st
    ON hb.hotel_id = st.hotel_id
  LEFT JOIN lead_time_summary lt
    ON hb.hotel_id = lt.hotel_id
  LEFT JOIN room_type_summary rt
    ON hb.hotel_id = rt.hotel_id
  LEFT JOIN discount_summary dc
    ON hb.hotel_id = dc.hotel_id
  LEFT JOIN latest_month lm
    ON hb.hotel_id = lm.hotel_id
    AND lm.month_rank = 1
),
-- SECTION 20: CALCULATE BUSINESS KPIs
hotel_kpis AS (
  SELECT
    hm.*,
    ROUND(
    100.0 * cancelled_bookings
    / NULLIF(total_bookings, 0),
    2
  ) AS cancellation_rate_pct,
    ROUND(
    100.0 * completed_bookings
    / NULLIF(total_bookings, 0),
    2
  ) AS completion_rate_pct,
    ROUND(
    100.0 * successful_collections
    / NULLIF(net_booking_revenue, 0),
```

```
    2
  ) AS payment_collection_pct,
    ROUND(
    100.0 * completed_room_nights_booked
    / NULLIF(total_rooms * 151, 0),
    2
  ) AS estimated_occupancy_pct,
    ROUND(
    net_booking_revenue
    / NULLIF(completed_bookings, 0),
    2
  ) AS revenue_per_completed_booking,
    ROUND(
    net_booking_revenue
    / NULLIF(total_rooms * 151, 0),
    2
  ) AS revenue_per_available_room_night,
    ROUND(
    net_booking_revenue
    + additional_service_revenue
    - total_operating_expenses,
    2
  ) AS operating_surplus_proxy,
    ROUND(
    100.0 * (
      net_booking_revenue
      + additional_service_revenue
      - total_operating_expenses
    ) / NULLIF(
      net_booking_revenue + additional_service_revenue,
      0
    ),
    2
  ) AS operating_surplus_margin_pct
  FROM hotel_metrics hm
)
-- SECTION 21: FINAL HOTEL PERFORMANCE REPORT
SELECT
  hotel_id,
  hotel_name,
  city,
  star_rating,
  hotel_type,
  total_bookings,
  completed_bookings,
  cancelled_bookings,
  unique_customers,
  total_guests,
  gross_booking_value,
  total_discounts,
  net_booking_revenue,
  average_booking_value,
  successful_collections,
  pending_payment_amount,
  total_refund_amount,
  payment_collection_pct,
  cancellation_rate_pct,
  completion_rate_pct,
  completed_room_nights_booked,
  estimated_occupancy_pct,
  revenue_per_completed_booking,
  revenue_per_available_room_night,
  additional_service_revenue,
  total_operating_expenses,
  utility_expenses,
  maintenance_expenses,
  staff_expenses,
  marketing_expenses,
  operating_surplus_proxy,
  operating_surplus_margin_pct,
  average_rating,
  total_reviews,
  low_rating_reviews,
  direct_bookings,
  ota_bookings,
  average_discount_pct,
  average_length_of_stay,
  average_lead_time_days,
  latest_revenue_month,
  latest_month_revenue,
  previous_month_revenue,
  latest_month_growth_pct,
  CASE
  WHEN cancellation_rate_pct >= 25
    THEN 'High Cancellation Risk'
  WHEN payment_collection_pct < 80
    THEN 'Payment Collection Needs Review'
  WHEN average_rating > 0 AND average_rating < 3.5
    THEN 'Customer Satisfaction Needs Improvement'
  WHEN operating_surplus_proxy < 0
```

```
    THEN 'Operating Loss Risk'
  WHEN estimated_occupancy_pct < 10
    THEN 'Low Estimated Occupancy'
  ELSE 'Monitor and Optimize'
END AS recommended_action
FROM hotel_kpis
ORDER BY
  operating_surplus_proxy DESC,
  net_booking_revenue DESC;
```

OUTPUT — 5 ROWS × 43 METRICS (TRANSPOSED: ONE COLUMN PER HOTEL)

metric	Mountain Escape	Grand Palace Hotel	City Business Hotel	Sea View Resort	Royal Comfort Inn
city	Manali	Mumbai	Bengaluru	Goa	Delhi
star_rating	4	5	4	4	3
hotel_type	Resort	Luxury	Business	Resort	Business
total_bookings	5	7	6	7	5
completed_bookings	5	6	6	4	4
cancelled_bookings	0	1	0	3	1
unique_customers	5	7	5	7	5
total_guests	17	14	12	20	11
gross_booking_value	80000	111000	66000	48000	22500
total_discounts	3500	7000	1000	2000	1000
net_booking_revenue	76500	104000	65000	46000	21500
average_booking_value	15300	17333.33	10833.33	11500	5375
successful_collections	59500	126000	65000	46000	14500
pending_payment_amount	18000	0	0	0	7000
total_refund_amount	0	24000	0	42000	9000
payment_collection_pct	77.78	121.15	100	100	67.44
cancellation_rate_pct	0	14.29	0	42.86	20
completion_rate_pct	100	85.71	100	57.14	80
completed_room_nights_booked	15	16	12	12	9
estimated_occupancy_pct	0.15	0.09	0.08	0.09	0.07
revenue_per_completed_booking	15300	17333.33	10833.33	11500	5375
revenue_per_available_room_night	7.79	5.74	4.3	3.38	1.78
additional_service_revenue	5100	10000	5200	4500	1600
total_operating_expenses	106000	142000	130000	115000	97000
utility_expenses	16000	25000	20000	18000	14000
maintenance_expenses	9000	12000	11000	10000	8000
staff_expenses	70000	90000	85000	75000	65000
marketing_expenses	11000	15000	14000	12000	10000
operating_surplus_proxy	-24400	-28000	-59800	-64500	-73900
operating_surplus_margin_pct	-29.9	-24.56	-85.19	-127.72	-319.91
average_rating	4.67	4.75	3.9	3.67	3.83
total_reviews	3	6	5	3	3
low_rating_reviews	0	0	0	0	0
direct_bookings	4	3	2	3	2
ota_bookings	0	3	3	4	2
average_discount_pct	3.89	6.32	1.52	4.17	4.17
average_length_of_stay	3	2.67	2	3	2.25
average_lead_time_days	9.8	9	9.67	10.14	10.8
latest_revenue_month	2025-05	2025-04	2025-05	2025-05	2025-05
latest_month_revenue	25000	23000	11000	11500	7000
previous_month_revenue	23000	22500	21500	11500	4800
latest_month_growth_pct	8.7	2.22	-48.84	0	45.83
recommended_action	**Payment Collection Needs Review**	**Operating Loss Risk**	**Operating Loss Risk**	**High Cancellation Risk**	**Payment Collection Needs Review**

READING THE RESULT

Grand Palace Hotel leads on bookings value (₹104,000 net) and guest rating (4.75), but its `payment_collection_pct` of 121.15 is impossible for a healthy ledger — it is the visible symptom of Defect 2 in Section 08 (a `Paid` payment recorded against a cancelled booking). Every hotel shows an operating deficit because four months of operating expenses are set against five months of bookings from a 30-row sample, so `operating_surplus_proxy` is a ranking device rather than a profit figure. Estimated occupancy of 0.07–0.15% reflects the denominator `total_rooms × 151`: the hotel table advertises 65–120 rooms but only five per hotel are modelled (Observation 7).

Question 2 — Profitability and cancellation risk

Joins each hotel through `rooms` and `booking_rooms` to its bookings, nets total expenses against completed gross revenue, and classifies the result: High Risk (cancellation rate of 30% or more and negative result), Cancellation Risk (20% or more), Expense Risk (negative result) or Healthy Performance.

QUESTION 2 — HOTEL PROFITABILITY & CANCELLATION RISK

```sql
WITH hotel_booking_metrics AS (
  SELECT
    h.hotel_id,
    h.hotel_name,
    h.city,
    h.star_rating,
    h.total_rooms,
    COUNT(b.booking_id) AS total_bookings,
    SUM(
    CASE
      WHEN b.booking_status = 'Completed' THEN 1
      ELSE 0
    END
  ) AS completed_bookings,
    SUM(
    CASE
      WHEN b.booking_status = 'Cancelled' THEN 1
      ELSE 0
    END
  ) AS cancelled_bookings,
    SUM(
    CASE
      WHEN b.booking_status = 'Completed'
        THEN COALESCE(b.total_amount, 0)
      ELSE 0
    END
  ) AS completed_booking_revenue
  FROM hotels h
  LEFT JOIN rooms r
    ON h.hotel_id = r.hotel_id
  LEFT JOIN booking_rooms br
    ON r.room_id = br.room_id
  LEFT JOIN bookings b
    ON br.booking_id = b.booking_id
  GROUP BY
    h.hotel_id,
    h.hotel_name,
    h.city,
    h.star_rating,
    h.total_rooms
),
hotel_expense_metrics AS (
  SELECT
    h.hotel_id,
    COALESCE(SUM(e.expense_amount), 0) AS total_expenses
  FROM hotels h
  LEFT JOIN expenses e
    ON h.hotel_id = e.hotel_id
  GROUP BY h.hotel_id
),
hotel_risk_analysis AS (
  SELECT
    hb.hotel_id,
    hb.hotel_name,
    hb.city,
    hb.star_rating,
    hb.total_rooms,
    hb.total_bookings,
    hb.completed_bookings,
    hb.cancelled_bookings,
    hb.completed_booking_revenue,
    he.total_expenses,
    ROUND(
    100.0 * hb.cancelled_bookings
    / NULLIF(hb.total_bookings, 0),
    2
  ) AS cancellation_rate_pct,
    ROUND(
```

```
    hb.completed_booking_revenue
    - he.total_expenses,
    2
  ) AS revenue_after_expenses_proxy
  FROM hotel_booking_metrics hb
  JOIN hotel_expense_metrics he
    ON hb.hotel_id = he.hotel_id
)
SELECT
  hotel_name,
  city,
  star_rating,
  total_bookings,
  completed_bookings,
  cancelled_bookings,
  completed_booking_revenue,
  total_expenses,
  cancellation_rate_pct,
  revenue_after_expenses_proxy,
  CASE
  WHEN cancellation_rate_pct >= 30
    AND revenue_after_expenses_proxy < 0
    THEN 'High Risk'
  WHEN cancellation_rate_pct >= 20
    THEN 'Cancellation Risk'
  WHEN revenue_after_expenses_proxy < 0
    THEN 'Expense Risk'
  ELSE 'Healthy Performance'
END AS hotel_performance_status
FROM hotel_risk_analysis
ORDER BY
  cancellation_rate_pct DESC,
  revenue_after_expenses_proxy ASC;
```

OUTPUT — 5 ROWS

hotel_name	city	star_rating	total_bookings	completed_bookings	cancelled_bookings
Sea View Resort	Goa	4	7	4	3
Royal Comfort Inn	Delhi	3	5	4	1
Grand Palace Hotel	Mumbai	5	7	6	1
City Business Hotel	Bengaluru	4	6	6	0
Mountain Escape	Manali	4	5	5	0

hotel_name	completed_booking_revenue	total_expenses	cancellation_rate_pct	revenue_after_expenses_proxy	hotel_performance_status
Sea View Resort	48000	115000	42.86	-67000	**High Risk**
Royal Comfort Inn	22500	97000	20	-74500	**Cancellation Risk**
Grand Palace Hotel	111000	142000	14.29	-31000	**Expense Risk**
City Business Hotel	66000	130000	0	-64000	**Expense Risk**
Mountain Escape	80000	106000	0	-26000	**Expense Risk**

KEY INSIGHT

Sea View Resort is the only High Risk property: three of its seven bookings were cancelled (42.86%) and it is ₹67,000 short after expenses. Royal Comfort Inn triggers Cancellation Risk at exactly 20%. The three remaining hotels are flagged Expense Risk because their expense postings exceed completed revenue.

Question 3 — Monthly revenue trend and growth

Aggregates 2025 bookings by month and uses `LAG()` to bring the previous month's revenue alongside, from which growth percentage and a Revenue Increased / Decreased label are derived. `NULLIF` protects the first month, which has no predecessor.

QUESTION 3 — REVENUE TRENDS & MONTH-OVER-MONTH GROWTH

```
WITH monthly_revenue AS (
  SELECT
    DATE_FORMAT(booking_date, '%Y-%m') AS booking_month,
    COUNT(*) AS total_bookings,
    SUM(
    CASE
      WHEN booking_status = 'Completed'
        THEN COALESCE(total_amount, 0)
      ELSE 0
    END
  ) AS completed_revenue,
    SUM(
    CASE
      WHEN booking_status = 'Completed' THEN 1
      ELSE 0
```

```sql
    END
  ) AS completed_bookings,
    SUM(
    CASE
      WHEN booking_status = 'Cancelled' THEN 1
      ELSE 0
    END
  ) AS cancelled_bookings
  FROM bookings
  WHERE booking_date >= '2025-01-01'
    AND booking_date < '2026-01-01'
  GROUP BY DATE_FORMAT(booking_date, '%Y-%m')
),
monthly_growth AS (
  SELECT
    booking_month,
    total_bookings,
    completed_revenue,
    completed_bookings,
    cancelled_bookings,
    LAG(completed_revenue) OVER (
    ORDER BY booking_month
  ) AS previous_month_revenue,
    LAG(total_bookings) OVER (
    ORDER BY booking_month
  ) AS previous_month_bookings
  FROM monthly_revenue
)
SELECT
  booking_month,
  total_bookings,
  completed_bookings,
  cancelled_bookings,
  ROUND(completed_revenue, 2) AS completed_revenue,
  ROUND(previous_month_revenue, 2) AS previous_month_revenue,
  ROUND(
  100.0 * (completed_revenue - previous_month_revenue)
  / NULLIF(previous_month_revenue, 0),
  2
) AS revenue_growth_pct,
  previous_month_bookings,
  CASE
  WHEN previous_month_revenue IS NULL
    THEN 'First Month'
  WHEN completed_revenue > previous_month_revenue
    THEN 'Revenue Increased'
  WHEN completed_revenue < previous_month_revenue
    THEN 'Revenue Decreased'
  ELSE 'No Change'
END AS revenue_trend
FROM monthly_growth
ORDER BY booking_month;
```

OUTPUT — 5 ROWS

booking_month	total_bookings	completed_bookings	cancelled_bookings	completed_revenue	previous_month_revenue	revenue_growth_pct	previous_month_bookings	revenue_trend
2025-01	6	5	1	51000	NULL	NULL	NULL	First Month
2025-02	6	5	1	75000	51000	47.06	6	**Revenue Increased**
2025-03	6	5	1	70000	75000	-6.67	6	**Revenue Decreased**
2025-04	6	5	1	75000	70000	7.14	6	**Revenue Increased**
2025-05	6	5	1	56500	75000	-24.67	6	**Revenue Decreased**

KEY INSIGHT

Volume is perfectly flat — six bookings, five completed and one cancelled in every month — so all of the movement comes from booking value. February jumps 47.06% to ₹75,000, April recovers to the same ₹75,000, and May falls 24.67% to ₹56,500, an average of ₹11,300 per completed booking.

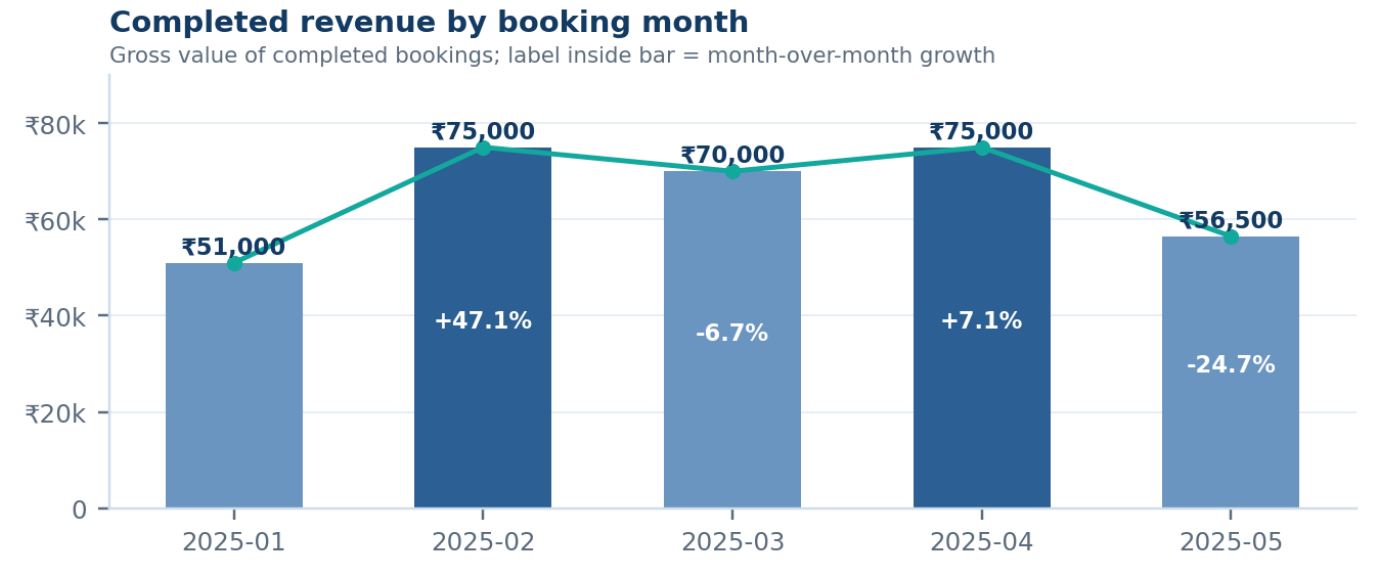


Figure 6.1 — Completed revenue by booking month (Question 3).

Question 4 — Booking source and channel performance

Rolls bookings up by channel, estimates commission as completed revenue multiplied by the channel rate, and ranks channels with `DENSE_RANK()`.

QUESTION 4 — BOOKING SOURCE & CHANNEL PERFORMANCE

```sql
WITH source_metrics AS (
  SELECT
    bs.booking_source_id,
    bs.source_name,
    bs.source_category,
    bs.commission_percentage,
    COUNT(b.booking_id) AS total_bookings,
    SUM(
    CASE
      WHEN b.booking_status = 'Completed' THEN 1
      ELSE 0
    END
  ) AS completed_bookings,
    SUM(
    CASE
      WHEN b.booking_status = 'Cancelled' THEN 1
      ELSE 0
    END
  ) AS cancelled_bookings,
    SUM(
    CASE
      WHEN b.booking_status = 'Completed'
        THEN COALESCE(b.total_amount, 0)
      ELSE 0
    END
  ) AS completed_revenue,
    AVG(
    CASE
      WHEN b.booking_status = 'Completed'
        THEN b.total_amount
    END
  ) AS average_booking_value
  FROM booking_sources bs
  LEFT JOIN bookings b
    ON bs.booking_source_id = b.booking_source_id
  GROUP BY
    bs.booking_source_id,
    bs.source_name,
    bs.source_category,
    bs.commission_percentage
),
source_analysis AS (
  SELECT
    *,
    ROUND(
    completed_revenue
    * COALESCE(commission_percentage, 0) / 100,
    2
  ) AS estimated_commission,
    ROUND(
    100.0 * cancelled_bookings
    / NULLIF(total_bookings, 0),
```

```sql
    2
  ) AS cancellation_rate_pct,
    ROUND(
    100.0 * completed_bookings
    / NULLIF(total_bookings, 0),
    2
  ) AS completion_rate_pct
  FROM source_metrics
)
SELECT
  source_name,
  source_category,
  commission_percentage,
  total_bookings,
  completed_bookings,
  cancelled_bookings,
  ROUND(completed_revenue, 2) AS completed_revenue,
  ROUND(average_booking_value, 2) AS average_booking_value,
  estimated_commission,
  cancellation_rate_pct,
  completion_rate_pct,
  DENSE_RANK() OVER (
  ORDER BY completed_revenue DESC
) AS revenue_rank,
  CASE
  WHEN cancellation_rate_pct >= 30
    THEN 'High Cancellation Risk'
  WHEN completed_revenue >= 50000
    THEN 'High Revenue Source'
  ELSE 'Needs Improvement'
END AS source_performance
FROM source_analysis
ORDER BY completed_revenue DESC;
```

OUTPUT — 5 ROWS

source_name	source_category	commission_percentage	total_bookings	completed_bookings	cancelled_bookings	completed_revenue
Hotel Website	Direct	0	7	7	0	89000
Mobile App	Direct	0	7	6	1	79000
MakeMyTrip	OTA	18	5	5	0	65500
Travel Agent	Agent	10	4	4	0	56000
Booking.com	OTA	15	7	3	4	38000

source_name	average_booking_value	estimated_commission	cancellation_rate_pct	completion_rate_pct	revenue_rank	source_performance
Hotel Website	12714.29	0	0	100	1	**High Revenue Source**
Mobile App	13166.67	0	14.29	85.71	2	**High Revenue Source**
MakeMyTrip	13100	11790	0	100	3	**High Revenue Source**
Travel Agent	14000	5600	0	100	4	**High Revenue Source**
Booking.com	12666.67	5700	57.14	42.86	5	**High Cancellation Risk**

KEY INSIGHT

The two commission-free direct channels deliver ₹168,000 of the ₹327,500 completed revenue (51.3%). Booking.com is the weakest channel on every measure: four of its seven bookings were cancelled (57.14%), leaving ₹38,000 of completed revenue after ₹5,700 of commission. MakeMyTrip is the most expensive channel in absolute terms at ₹11,790 but converts every booking it sends.

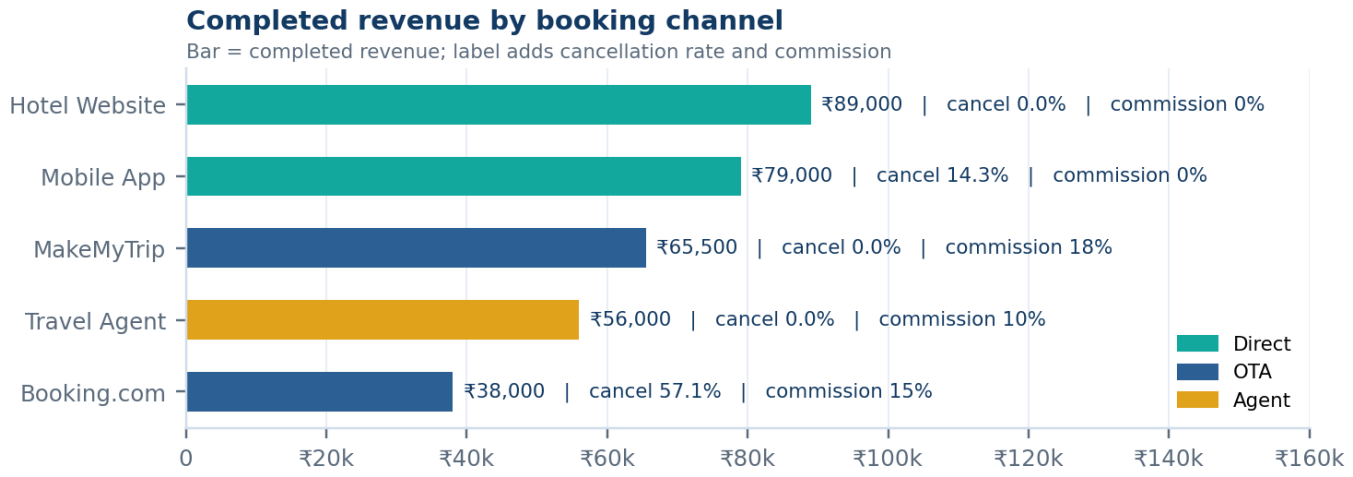


Figure 6.2 — Booking channel economics (Question 4).

Question 5 — Customer lifetime value and repeat behaviour

Builds a per-guest profile — bookings, completions, cancellations, lifetime revenue, first and latest stay, and the span between them — then ranks guests by revenue and assigns a value tier (30,000 or more is High, 15,000 or more Medium) and a behaviour label (Repeat, One-Time, No Completed Booking).

QUESTION 5 — CUSTOMER LIFETIME VALUE & REPEAT BOOKING

```
WITH customer_metrics AS (
  SELECT
    c.customer_id,
    CONCAT(
    c.first_name,
    ' ',
    COALESCE(c.last_name, '')
  ) AS customer_full_name,
    c.city,
    c.country,
    c.customer_type AS registered_customer_type,
    COUNT(b.booking_id) AS total_bookings,
    SUM(
    CASE
      WHEN b.booking_status = 'Completed' THEN 1
      ELSE 0
    END
  ) AS completed_bookings,
    SUM(
    CASE
      WHEN b.booking_status = 'Cancelled' THEN 1
      ELSE 0
    END
  ) AS cancelled_bookings,
    SUM(
    CASE
      WHEN b.booking_status = 'Completed'
        THEN COALESCE(b.total_amount, 0)
      ELSE 0
    END
  ) AS customer_lifetime_revenue,
    MIN(
    CASE
      WHEN b.booking_status = 'Completed'
        THEN b.booking_date
    END
  ) AS first_completed_booking,
    MAX(
    CASE
      WHEN b.booking_status = 'Completed'
        THEN b.booking_date
    END
  ) AS latest_completed_booking
  FROM customers c
  LEFT JOIN bookings b
    ON c.customer_id = b.customer_id
  GROUP BY
    c.customer_id,
    c.first_name,
    c.last_name,
    c.city,
    c.country,
    c.customer_type
),
```

```sql
customer_analysis AS (
  SELECT
    *,
    ROUND(
    customer_lifetime_revenue
    / NULLIF(completed_bookings, 0),
    2
  ) AS average_booking_value,
    DATEDIFF(
    latest_completed_booking,
    first_completed_booking
  ) AS customer_booking_span_days
  FROM customer_metrics
)
SELECT
  customer_id,
  customer_full_name,
  city,
  country,
  registered_customer_type,
  total_bookings,
  completed_bookings,
  cancelled_bookings,
  ROUND(customer_lifetime_revenue, 2)
  AS customer_lifetime_revenue,
  average_booking_value,
  first_completed_booking,
  latest_completed_booking,
  customer_booking_span_days,
  DENSE_RANK() OVER (
  ORDER BY customer_lifetime_revenue DESC
) AS customer_revenue_rank,
  CASE
  WHEN customer_lifetime_revenue >= 30000
    THEN 'High Value Customer'
  WHEN customer_lifetime_revenue >= 15000
    THEN 'Medium Value Customer'
  ELSE 'Low Value Customer'
END AS customer_value_segment,
  CASE
  WHEN completed_bookings >= 2
    THEN 'Repeat Customer'
  WHEN completed_bookings = 1
    THEN 'One-Time Customer'
  ELSE 'No Completed Booking'
END AS booking_behavior
FROM customer_analysis
ORDER BY
  customer_lifetime_revenue DESC,
  completed_bookings DESC;
```

OUTPUT — 20 ROWS

customer_id	customer_full_name	city	country	registered_customer_type	total_bookings	completed_bookings	cancelled_bookings	customer_lifetime_revenue
3	Rahul Verma	Delhi	India	Regular	2	2	0	36000
8	Isha Kapoor	Chandigarh	India	Regular	2	2	0	27000
7	Vikram Singh	Jaipur	India	Premium	2	2	0	24000
12	Kavya Iyer	Chennai	India	Premium	1	1	0	24000
17	Karan Desai	Surat	India	Premium	1	1	0	24000
20	Simran Kaur	Amritsar	India	Regular	1	1	0	24000
4	Sneha Rao	Bengaluru	India	Premium	2	2	0	22000
9	Rohan Nair	Kochi	India	Premium	2	2	0	18500
10	Meera Joshi	Nashik	India	Regular	2	1	1	18000
14	Neha Kulkarni	Pune	India	Premium	1	1	0	18000
2	Priya Patel	Mumbai	India	Premium	2	1	1	15000
1	Aarav Shah	Mumbai	India	Regular	2	2	0	13000
5	Kabir Mehta	Pune	India	Regular	2	2	0	13000
11	Aditya Gupta	Lucknow	India	Regular	1	1	0	12000
18	Riya Fernandes	Goa	India	Regular	1	1	0	12000
13	Arjun Malhotra	Mumbai	India	Regular	1	1	0	11000
19	Yash Chopra	Noida	India	Premium	1	1	0	11000
15	Dev Khanna	Delhi	India	Regular	1	1	0	5000
6	Ananya Das	Kolkata	India	Regular	2	0	2	0
16	Pooja Sharma	Jaipur	India	Regular	1	0	1	0

customer_id	customer_full_name	average_booking_value	first_completed_booking	latest_completed_booking	customer_booking_span_days	customer_revenue_rank	customer_value_segment	booking_behavior
3	Rahul Verma	18000	2025-01-10	2025-04-20	100	1	High Value Customer	Repeat Customer
8	Isha Kapoor	13500	2025-02-05	2025-05-15	99	2	Medium Value Customer	Repeat Customer
7	Vikram Singh	12000	2025-02-01	2025-05-10	98	3	Medium Value Customer	Repeat Customer
12	Kavya Iyer	24000	2025-02-25	2025-02-25	0	3	Medium Value Customer	One-Time Customer
17	Karan Desai	24000	2025-03-20	2025-03-20	0	3	Medium Value Customer	One-Time Customer
20	Simran Kaur	24000	2025-04-05	2025-04-05	0	3	Medium Value Customer	One-Time Customer
4	Sneha Rao	11000	2025-01-15	2025-04-25	100	4	Medium Value Customer	Repeat Customer
9	Rohan Nair	9250	2025-02-10	2025-05-20	99	5	Medium Value Customer	Repeat Customer
10	Meera Joshi	18000	2025-05-25	2025-05-25	0	6	Medium Value Customer	One-Time Customer
14	Neha Kulkarni	18000	2025-03-05	2025-03-05	0	6	Medium Value Customer	One-Time Customer
2	Priya Patel	15000	2025-01-08	2025-01-08	0	7	Medium Value Customer	One-Time Customer
1	Aarav Shah	6500	2025-01-05	2025-04-10	95	8	Low Value Customer	Repeat Customer
5	Kabir Mehta	6500	2025-01-20	2025-05-01	101	8	Low Value Customer	Repeat Customer
11	Aditya Gupta	12000	2025-02-20	2025-02-20	0	9	Low Value Customer	One-Time Customer
18	Riya Fernandes	12000	2025-03-25	2025-03-25	0	9	Low Value Customer	One-Time Customer
13	Arjun Malhotra	11000	2025-03-01	2025-03-01	0	10	Low Value Customer	One-Time Customer
19	Yash Chopra	11000	2025-04-01	2025-04-01	0	10	Low Value Customer	One-Time Customer

customer_id	customer_full_name	average_booking_value	first_completed_booking	latest_completed_booking	customer_booking_span_days	customer_revenue_rank	customer_value_segment	booking_behavior
15	Dev Khanna	5000	2025-03-10	2025-03-10	0	11	Low Value Customer	One-Time Customer
6	Ananya Das	NULL	NULL	NULL	NULL	12	Low Value Customer	No Completed Booking
16	Pooja Sharma	NULL	NULL	NULL	NULL	12	Low Value Customer	No Completed Booking

KEY INSIGHT

Only one guest, Rahul Verma (₹36,000 over two stays), reaches the High Value tier; ten are Medium and nine Low. Seven guests are repeat customers, eleven have stayed once and two — Ananya Das and Pooja Sharma — have only cancelled bookings. `DENSE_RANK()` gives three guests the shared rank 3 (₹24,000 each) and continues at 4 with no gap, unlike `RANK()`.

SECTION

07 Operations, Risk and Guest Analytics

Questions 6–15: room-type demand, cancellation exposure, satisfaction, cost efficiency, booking lead time, utilisation, collections, services, hotel ranking and the group summary.

Question 6 — Room type revenue and demand

Resolves every room to its type and aggregates bookings and revenue per type, using `COUNT(DISTINCT ...)` so a booking is counted once even if it spans several rooms.

QUESTION 6 — ROOM TYPE REVENUE & DEMAND

```sql
WITH room_type_metrics AS (
  SELECT
    rt.room_type_id,
    rt.room_type_name,
    rt.base_price,
    rt.max_guests,
    COUNT(DISTINCT r.room_id) AS total_rooms,
    COUNT(DISTINCT b.booking_id) AS total_bookings,
    COUNT(
    DISTINCT CASE
      WHEN b.booking_status = 'Completed'
        THEN b.booking_id
    END
  ) AS completed_bookings,
    COUNT(
    DISTINCT CASE
      WHEN b.booking_status = 'Cancelled'
        THEN b.booking_id
    END
  ) AS cancelled_bookings,
    SUM(
    CASE
      WHEN b.booking_status = 'Completed'
        THEN COALESCE(b.total_amount, 0)
      ELSE 0
    END
  ) AS completed_revenue
  FROM room_types rt
  LEFT JOIN rooms r
    ON rt.room_type_id = r.room_type_id
  LEFT JOIN booking_rooms br
    ON r.room_id = br.room_id
  LEFT JOIN bookings b
    ON br.booking_id = b.booking_id
  GROUP BY
    rt.room_type_id,
    rt.room_type_name,
    rt.base_price,
    rt.max_guests
),
room_type_analysis AS (
  SELECT
    *,
    ROUND(
    completed_revenue
    / NULLIF(completed_bookings, 0),
    2
  ) AS average_revenue_per_booking,
    ROUND(
    100.0 * completed_bookings
    / NULLIF(total_bookings, 0),
    2
  ) AS completion_rate_pct
  FROM room_type_metrics
)
SELECT
  room_type_name,
  base_price,
  max_guests,
  total_rooms,
  total_bookings,
  completed_bookings,
  cancelled_bookings,
  ROUND(completed_revenue, 2) AS completed_revenue,
  average_revenue_per_booking,
  completion_rate_pct,
  DENSE_RANK() OVER (
  ORDER BY completed_revenue DESC
) AS revenue_rank,
```

```
  CASE
  WHEN completed_revenue >= 50000
    THEN 'High Revenue Room Type'
  WHEN completed_revenue >= 20000
    THEN 'Medium Revenue Room Type'
  ELSE 'Low Revenue Room Type'
END AS revenue_category
FROM room_type_analysis
ORDER BY completed_revenue DESC;
```

OUTPUT — 5 ROWS

room_type_name	base_price	max_guests	total_rooms	total_bookings	completed_bookings
Suite	8000	4	5	6	6
Executive	5500	2	5	5	5
Family Room	6000	5	5	6	4
Deluxe	4000	3	5	7	5
Standard	2500	2	5	6	5

room_type_name	cancelled_bookings	completed_revenue	average_revenue_per_booking	completion_rate_pct	revenue_rank	revenue_category
Suite	0	86000	14333.33	100	1	High Revenue Room Type
Executive	0	78500	15700	100	2	High Revenue Room Type
Family Room	2	58000	14500	66.67	3	High Revenue Room Type
Deluxe	2	57000	11400	71.43	4	High Revenue Room Type
Standard	1	48000	9600	83.33	5	Medium Revenue Room Type

KEY INSIGHT

Suites lead with ₹86,000 from six completed stays, followed by Executive rooms at ₹78,500. Family and Deluxe rooms each lose two bookings to cancellation, giving them the lowest completion rates (66.67% and 71.43%). Standard rooms are the only type in the Medium revenue band.

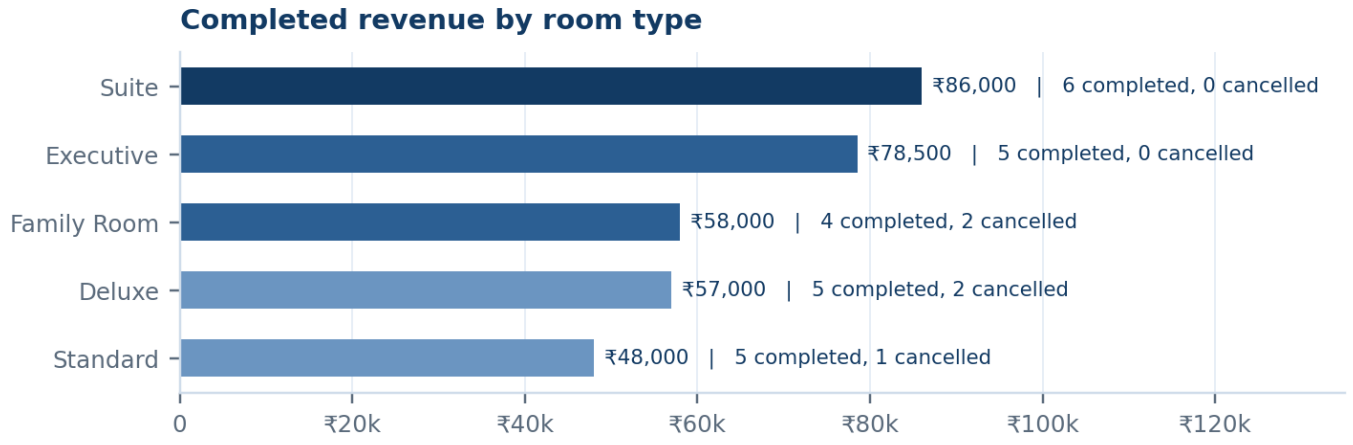


Figure 7.1 — Completed revenue by room type (Question 6).

Question 7 — Cancellation and refund risk

Combines three summaries — cancellation counts, refund totals and a de-duplicated list of reasons built with `GROUP_CONCAT(DISTINCT ...)` — and bands each hotel as High (30% or more), Medium (15% or more) or Low cancellation risk.

QUESTION 7 — CANCELLATION & REFUND RISK

```
WITH cancellation_summary AS (
  SELECT
    b.booking_id,
    b.booking_status,
    b.total_amount,
    b.booking_source_id,
    h.hotel_id,
    h.hotel_name,
    c.cancellation_reason,
    COALESCE(c.refund_amount, 0) AS refund_amount
  FROM bookings b
  JOIN booking_rooms br
```

```sql
    ON b.booking_id = br.booking_id
  JOIN rooms r
    ON br.room_id = r.room_id
  JOIN hotels h
    ON r.hotel_id = h.hotel_id
  LEFT JOIN cancellations c
    ON b.booking_id = c.booking_id
),
hotel_risk AS (
  SELECT
    hotel_id,
    hotel_name,
    COUNT(DISTINCT booking_id) AS total_bookings,
    COUNT(DISTINCT CASE
      WHEN booking_status = 'Cancelled'
        THEN booking_id
      END) AS cancelled_bookings,
    SUM(DISTINCT 0) AS placeholder
  FROM cancellation_summary
  GROUP BY hotel_id, hotel_name
),
refund_summary AS (
  SELECT
    hotel_id,
    hotel_name,
    SUM(CASE
      WHEN booking_status = 'Cancelled'
        THEN refund_amount
      ELSE 0
      END) AS total_refund_amount
  FROM (
    SELECT DISTINCT
      booking_id,
      hotel_id,
      hotel_name,
      booking_status,
      refund_amount
    FROM cancellation_summary
  )x
  GROUP BY hotel_id, hotel_name
),
reason_summary AS (
  SELECT
    hotel_id,
    hotel_name,
    COUNT(*) AS cancellation_records,
    GROUP_CONCAT(
    DISTINCT cancellation_reason
    ORDER BY cancellation_reason
      SEPARATOR ', '
  ) AS cancellation_reasons
  FROM (
    SELECT DISTINCT
      booking_id,
      hotel_id,
      hotel_name,
      cancellation_reason
    FROM cancellation_summary
    WHERE cancellation_reason IS NOT NULL
  )y
  GROUP BY hotel_id, hotel_name
)
SELECT
  hr.hotel_name,
  hr.total_bookings,
  hr.cancelled_bookings,
  ROUND(
  hr.cancelled_bookings * 100.0 /
  NULLIF(hr.total_bookings, 0), 2
) AS cancellation_rate_pct,
  rs.total_refund_amount,
  COALESCE(re.cancellation_records, 0) AS cancellation_records,
  COALESCE(re.cancellation_reasons, 'No recorded reason')
  AS cancellation_reasons,
  CASE
  WHEN hr.cancelled_bookings * 100.0 /
  NULLIF(hr.total_bookings, 0) >= 30
    THEN 'High Cancellation Risk'
  WHEN hr.cancelled_bookings * 100.0 /
  NULLIF(hr.total_bookings, 0) >= 15
    THEN 'Medium Cancellation Risk'
  ELSE 'Low Cancellation Risk'
END AS cancellation_risk
FROM hotel_risk hr
JOIN refund_summary rs
  ON hr.hotel_id = rs.hotel_id
LEFT JOIN reason_summary re
  ON hr.hotel_id = re.hotel_id
ORDER BY
  cancellation_rate_pct DESC,
```

```
    total_refund_amount DESC;
```

OUTPUT — 5 ROWS

hotel_name	total_bookings	cancelled_bookings	cancellation_rate_pct	total_refund_amount	cancellation_records	cancellation_reasons	cancellation_risk
Sea View Resort	7	3	42.86	42000	3	Change in travel plan, Date change required, Unexpected emergency	High Cancellation Risk
Royal Comfort Inn	5	1	20	9000	1	Booking made by mistake	Medium Cancellation Risk
Grand Palace Hotel	7	1	14.29	24000	1	Personal reasons	Low Cancellation Risk
Mountain Escape	5	0	0	0	0	No recorded reason	Low Cancellation Risk
City Business Hotel	6	0	0	0	0	No recorded reason	Low Cancellation Risk

KEY INSIGHT

Sea View Resort carries ₹42,000 of the ₹75,000 refunded across the group. Two hotels, Mountain Escape and City Business Hotel, have no cancellations at all and report “No recorded reason” through the `COALESCE` default.

Question 8 — Guest review and satisfaction

Counts positive (4 or more), neutral (3 to under 4) and negative (under 3) reviews per hotel, converts them to a positive-review percentage and assigns a satisfaction label from the average rating.

QUESTION 8 — GUEST REVIEW & SATISFACTION

```
WITH hotel_reviews AS (
  SELECT
    h.hotel_id,
    h.hotel_name,
    rv.review_id,
    rv.rating,
    rv.review_text
  FROM reviews rv
  JOIN bookings b
    ON rv.booking_id = b.booking_id
  JOIN booking_rooms br
    ON b.booking_id = br.booking_id
  JOIN rooms r
    ON br.room_id = r.room_id
  JOIN hotels h
    ON r.hotel_id = h.hotel_id
),
review_summary AS (
  SELECT
    hotel_id,
    hotel_name,
    COUNT(DISTINCT review_id) AS total_reviews,
    ROUND(AVG(rating), 2) AS average_rating,
    SUM(CASE WHEN rating >= 4 THEN 1 ELSE 0 END)
    AS positive_reviews,
    SUM(CASE WHEN rating < 3 THEN 1 ELSE 0 END)
    AS negative_reviews,
    SUM(CASE WHEN rating >= 3 AND rating < 4
        THEN 1 ELSE 0 END) AS neutral_reviews
  FROM hotel_reviews
  GROUP BY hotel_id, hotel_name
)
SELECT
  hotel_name,
  total_reviews,
  average_rating,
  positive_reviews,
  neutral_reviews,
  negative_reviews,
  ROUND(
  positive_reviews * 100.0 /
  NULLIF(total_reviews, 0), 2
) AS positive_review_pct,
  CASE
  WHEN average_rating >= 4.5
    THEN 'Excellent Satisfaction'
  WHEN average_rating >= 4.0
    THEN 'Good Satisfaction'
  WHEN average_rating >= 3.0
    THEN 'Average Satisfaction'
```

```
    ELSE 'Needs Improvement'
  END AS satisfaction_level
  FROM review_summary
  ORDER BY average_rating DESC, total_reviews DESC;
```

OUTPUT — 5 ROWS

hotel_name	total_reviews	average_rating	positive_reviews	neutral_reviews	negative_reviews	positive_review_pct	satisfaction_level
Grand Palace Hotel	6	4.75	6	0	0	100	Excellent Satisfaction
Mountain Escape	3	4.67	3	0	0	100	Excellent Satisfaction
City Business Hotel	5	3.9	3	2	0	60	Average Satisfaction
Royal Comfort Inn	3	3.83	2	1	0	66.67	Average Satisfaction
Sea View Resort	3	3.67	2	1	0	66.67	Average Satisfaction

KEY INSIGHT

Grand Palace Hotel and Mountain Escape are Excellent (4.75 and 4.67) with every review positive. The other three hotels are Average at 3.67–3.90, and no review anywhere falls below 3.0.

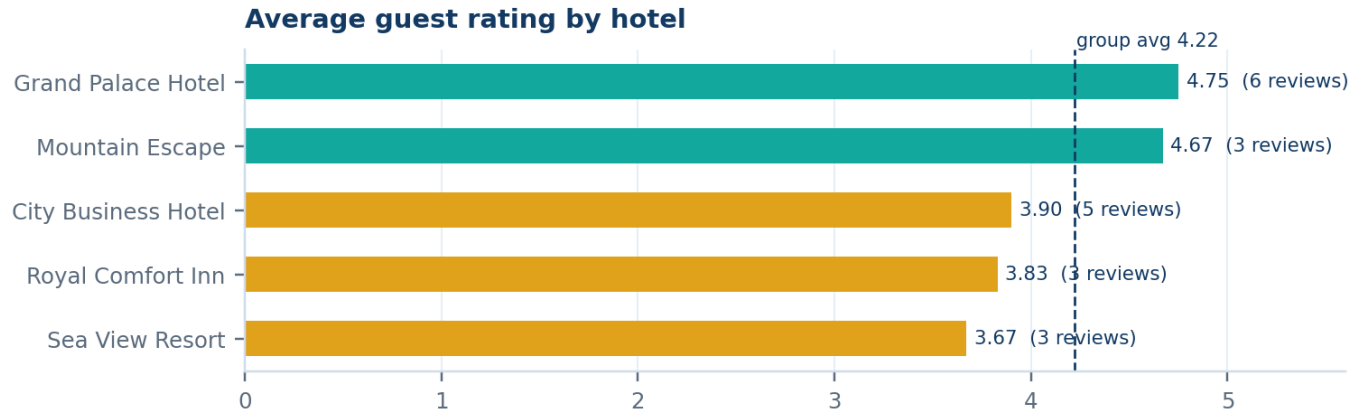


Figure 7.2 — Average guest rating by hotel (Question 8).

Question 9 — Operating expense and cost efficiency

Sets each hotel's total expenses against completed gross revenue and classifies the ratio as High (40% or less), Moderate (70% or less) or Low cost efficiency.

QUESTION 9 — OPERATING EXPENSES & COST EFFICIENCY

```
WITH revenue_summary AS (
  SELECT
    h.hotel_id,
    h.hotel_name,
    SUM(
    CASE
      WHEN b.booking_status = 'Completed'
        THEN b.total_amount
      ELSE 0
    END
  ) AS completed_revenue,
    COUNT(
    DISTINCT CASE
      WHEN b.booking_status = 'Completed'
        THEN b.booking_id
    END
  ) AS completed_bookings
  FROM hotels h
  LEFT JOIN rooms r
    ON h.hotel_id = r.hotel_id
  LEFT JOIN booking_rooms br
    ON r.room_id = br.room_id
  LEFT JOIN bookings b
    ON br.booking_id = b.booking_id
  GROUP BY h.hotel_id, h.hotel_name
),
expense_summary AS (
  SELECT
    hotel_id,
    SUM(expense_amount) AS total_expenses,
```

```
    COUNT(*) AS expense_records
  FROM expenses
  GROUP BY hotel_id
)
SELECT
  h.hotel_name,
  COALESCE(rs.completed_bookings, 0) AS completed_bookings,
  COALESCE(rs.completed_revenue, 0) AS completed_revenue,
  COALESCE(es.total_expenses, 0) AS total_operating_expenses,
  COALESCE(es.expense_records, 0) AS expense_records,
  COALESCE(rs.completed_revenue, 0)
  - COALESCE(es.total_expenses, 0)
  AS revenue_after_expenses_proxy,
  ROUND(
  COALESCE(es.total_expenses, 0) * 100.0 /
  NULLIF(rs.completed_revenue, 0),
  2
) AS expense_to_revenue_pct,
  CASE
  WHEN COALESCE(rs.completed_revenue, 0) = 0
    THEN 'No Revenue'
  WHEN COALESCE(es.total_expenses, 0) * 100.0 /
  NULLIF(rs.completed_revenue, 0) <= 40
    THEN 'High Cost Efficiency'
  WHEN COALESCE(es.total_expenses, 0) * 100.0 /
  NULLIF(rs.completed_revenue, 0) <= 70
    THEN 'Moderate Cost Efficiency'
  ELSE 'Low Cost Efficiency'
END AS cost_efficiency_status
FROM hotels h
LEFT JOIN revenue_summary rs
  ON h.hotel_id = rs.hotel_id
LEFT JOIN expense_summary es
  ON h.hotel_id = es.hotel_id
ORDER BY expense_to_revenue_pct DESC;
```

OUTPUT — 5 ROWS

hotel_name	completed_bookings	completed_revenue	total_operating_expenses	expense_records	revenue_after_expenses_proxy	expense_to_revenue_pct	cost_efficiency_status
Royal Comfort Inn	4	22500	97000	4	-74500	431.11	Low Cost Efficiency
Sea View Resort	4	48000	115000	4	-67000	239.58	Low Cost Efficiency
City Business Hotel	6	66000	130000	4	-64000	196.97	Low Cost Efficiency
Mountain Escape	5	80000	106000	4	-26000	132.5	Low Cost Efficiency
Grand Palace Hotel	6	111000	142000	4	-31000	127.93	Low Cost Efficiency

KEY INSIGHT

All five hotels land in Low Cost Efficiency, with expense-to-revenue ratios from 127.93% (Grand Palace) to 431.11% (Royal Comfort Inn). As with Question 1, this reflects the scale of the sample — four months of full operating cost against thirty bookings — so the ordering is informative while the absolute ratios are not.

Question 10 — Booking lead time

Buckets each booking by days between booking date and check-in: Last-Minute (up to 3), Short-Advance (up to 14), Medium-Advance (up to 30) or Early, then summarises volume, value and cancellation by bucket.

QUESTION 10 — BOOKING LEAD TIME & BEHAVIOUR

```
WITH booking_lead_time AS (
  SELECT
    booking_id,
    hotel_id,
    booking_date,
    check_in_date,
    booking_status,
    total_amount,
    DATEDIFF(check_in_date, booking_date) AS lead_time_days,
    CASE
    WHEN DATEDIFF(check_in_date, booking_date) <= 3
      THEN 'Last-Minute Booking'
    WHEN DATEDIFF(check_in_date, booking_date) <= 14
      THEN 'Short-Advance Booking'
    WHEN DATEDIFF(check_in_date, booking_date) <= 30
      THEN 'Medium-Advance Booking'
    ELSE 'Early Booking'
  END AS booking_group
  FROM bookings
),
```

```sql
lead_time_summary AS (
  SELECT
    booking_group,
    COUNT(*) AS total_bookings,
    SUM(CASE
      WHEN booking_status = 'Completed' THEN 1
      ELSE 0
      END) AS completed_bookings,
    SUM(CASE
      WHEN booking_status = 'Cancelled' THEN 1
      ELSE 0
      END) AS cancelled_bookings,
    ROUND(AVG(lead_time_days), 2) AS average_lead_time_days,
    ROUND(AVG(total_amount), 2) AS average_booking_value,
    SUM(CASE
      WHEN booking_status = 'Completed' THEN total_amount
      ELSE 0
      END) AS completed_revenue
  FROM booking_lead_time
  GROUP BY booking_group
)
SELECT
  booking_group,
  total_bookings,
  completed_bookings,
  cancelled_bookings,
  average_lead_time_days,
  average_booking_value,
  completed_revenue,
  ROUND(
  cancelled_bookings * 100.0 /
  NULLIF(total_bookings, 0), 2
) AS cancellation_rate_pct,
  CASE
  WHEN cancelled_bookings * 100.0 /
  NULLIF(total_bookings, 0) >= 30
    THEN 'High Cancellation Risk'
  WHEN cancelled_bookings * 100.0 /
  NULLIF(total_bookings, 0) >= 15
    THEN 'Medium Cancellation Risk'
  ELSE 'Low Cancellation Risk'
END AS cancellation_risk
FROM lead_time_summary
ORDER BY average_lead_time_days;
```

OUTPUT — 1 ROW

booking_group	total_bookings	completed_bookings	cancelled_bookings	average_lead_time_days	average_booking_value	completed_revenue	cancellation_rate_pct	cancellation_risk
Short-Advance Booking	30	25	5	9.83	13416.67	327500	16.67	Medium Cancellation Risk

OBSERVATION

All thirty bookings were made between 5 and 12 days before check-in, so every row lands in the single Short-Advance bucket. The query is correct, but the thresholds cannot separate this dataset (Observation 9 in Section 08).

Question 11 — Room utilisation

A single aggregate over hotels, rooms, booking rooms and bookings: the share of each hotel's rooms that have at least one completed booking, banded High (80% or more), Moderate (50% or more) or Low.

QUESTION 11 — ROOM OCCUPANCY & AVAILABILITY

```sql
SELECT
  h.hotel_name,
  COUNT(DISTINCT r.room_id) AS total_rooms,
  COUNT(DISTINCT CASE
    WHEN b.booking_status = 'Completed'
      THEN r.room_id
    END) AS rooms_with_completed_bookings,
  COUNT(DISTINCT CASE
    WHEN b.booking_status = 'Cancelled'
      THEN r.room_id
    END) AS rooms_with_cancelled_bookings,
  ROUND(
  COUNT(DISTINCT CASE
      WHEN b.booking_status = 'Completed'
        THEN r.room_id
      END) * 100.0 /
  NULLIF(COUNT(DISTINCT r.room_id), 0),
  2
) AS room_utilization_pct,
  CASE
  WHEN COUNT(DISTINCT CASE
```

```
      WHEN b.booking_status = 'Completed'
        THEN r.room_id
      END) * 100.0 /
  NULLIF(COUNT(DISTINCT r.room_id), 0) >= 80
    THEN 'High Utilization'
  WHEN COUNT(DISTINCT CASE
      WHEN b.booking_status = 'Completed'
        THEN r.room_id
      END) * 100.0 /
  NULLIF(COUNT(DISTINCT r.room_id), 0) >= 50
    THEN 'Moderate Utilization'
  ELSE 'Low Utilization'
END AS utilization_status
FROM hotels h
LEFT JOIN rooms r
  ON h.hotel_id = r.hotel_id
LEFT JOIN booking_rooms br
  ON r.room_id = br.room_id
LEFT JOIN bookings b
  ON br.booking_id = b.booking_id
GROUP BY h.hotel_id, h.hotel_name
ORDER BY room_utilization_pct DESC;
```

OUTPUT — 5 ROWS

hotel_name	total_rooms	rooms_with_completed_bookings	rooms_with_cancelled_bookings	room_utilization_pct	utilization_status
Grand Palace Hotel	5	5	1	100	**High Utilization**
Mountain Escape	5	5	0	100	**High Utilization**
City Business Hotel	5	5	0	100	**High Utilization**
Sea View Resort	5	4	3	80	**High Utilization**
Royal Comfort Inn	5	4	1	80	**High Utilization**

KEY INSIGHT

Three hotels have every room booked at least once (100%); Sea View Resort and Royal Comfort Inn reach 80%. Note this measures how many rooms were ever sold, not the share of nights occupied.

Question 12 — Payment collection and outstanding revenue

Pre-aggregates payments per booking, then rolls collected, pending and refunded amounts up to hotel level against the total booking value and bands the collection rate: High (90% or more), Moderate (60% or more), Low.

QUESTION 12 — PAYMENT COLLECTION & OUTSTANDING REVENUE

```
WITH payment_summary AS (
  SELECT
    booking_id,
    COUNT(*) AS payment_records,
    SUM(CASE
      WHEN payment_status = 'Paid'
        THEN COALESCE(payment_amount, 0)
      ELSE 0
      END) AS collected_amount,
    SUM(CASE
      WHEN payment_status = 'Pending'
        THEN COALESCE(payment_amount, 0)
      ELSE 0
      END) AS pending_amount,
    SUM(CASE
      WHEN payment_status = 'Refunded'
        THEN COALESCE(payment_amount, 0)
      ELSE 0
      END) AS refunded_amount
  FROM payments
  GROUP BY booking_id
)
SELECT
  h.hotel_name,
  COUNT(DISTINCT b.booking_id) AS total_bookings,
  SUM(COALESCE(ps.payment_records, 0)) AS payment_records,
  SUM(COALESCE(ps.collected_amount, 0)) AS total_collected,
  SUM(COALESCE(ps.pending_amount, 0)) AS total_pending,
  SUM(COALESCE(ps.refunded_amount, 0)) AS total_refunded,
  SUM(COALESCE(b.total_amount, 0)) AS total_booking_value,
  SUM(COALESCE(b.total_amount, 0))
  - SUM(COALESCE(ps.collected_amount, 0))
  AS booking_value_not_marked_paid,
  ROUND(
  SUM(COALESCE(ps.collected_amount, 0)) * 100.0 /
  NULLIF(SUM(COALESCE(b.total_amount, 0)), 0),
  2
) AS collection_rate_pct,
```

```sql
    CASE
    WHEN SUM(COALESCE(b.total_amount, 0)) = 0
      THEN 'No Booking Value'
    WHEN SUM(COALESCE(ps.collected_amount, 0)) * 100.0 /
    NULLIF(SUM(COALESCE(b.total_amount, 0)), 0) >= 90
      THEN 'High Collection'
    WHEN SUM(COALESCE(ps.collected_amount, 0)) * 100.0 /
    NULLIF(SUM(COALESCE(b.total_amount, 0)), 0) >= 60
      THEN 'Moderate Collection'
    ELSE 'Low Collection'
  END AS collection_status
  FROM hotels h
  LEFT JOIN bookings b
    ON h.hotel_id = b.hotel_id
  LEFT JOIN payment_summary ps
    ON b.booking_id = ps.booking_id
  GROUP BY h.hotel_id, h.hotel_name
  ORDER BY collection_rate_pct DESC;
```

OUTPUT — 5 ROWS

hotel_name	total_bookings	payment_records	total_collected	total_pending	total_refunded
City Business Hotel	6	6	65000	0	0
Grand Palace Hotel	7	7	126000	0	0
Mountain Escape	5	5	59500	18000	0
Sea View Resort	7	7	46000	0	0
Royal Comfort Inn	5	5	14500	7000	0

hotel_name	total_booking_value	booking_value_not_marked_paid	collection_rate_pct	collection_status
City Business Hotel	66000	1000	98.48	High Collection
Grand Palace Hotel	135000	9000	93.33	High Collection
Mountain Escape	80000	20500	74.38	Moderate Collection
Sea View Resort	90000	44000	51.11	Low Collection
Royal Comfort Inn	31500	17000	46.03	Low Collection

KEY INSIGHT

City Business Hotel and Grand Palace Hotel collect over 90% of booking value. Mountain Escape has ₹18,000 pending on booking 30 and Royal Comfort Inn ₹7,000 pending on booking 29. The low rates at Sea View Resort and Royal Comfort Inn are partly structural: `total_booking_value` includes cancelled bookings at gross value, which are by definition not collected.

Question 13 — Additional services and guest spending

Aggregates service consumption per service — usage records, quantity, distinct bookings and revenue — ranks services with `DENSE_RANK()` and bands them by revenue.

QUESTION 13 — ADDITIONAL SERVICES & SPENDING

```sql
WITH service_summary AS (
  SELECT
    s.service_id,
    s.service_name,
    s.service_category,
    s.service_price,
    COUNT(su.usage_id) AS total_usage_records,
    SUM(COALESCE(su.quantity, 0)) AS total_quantity,
    SUM(COALESCE(su.total_cost, 0)) AS total_service_revenue,
    COUNT(DISTINCT su.booking_id) AS unique_bookings
  FROM services s
  LEFT JOIN service_usage su
    ON s.service_id = su.service_id
  GROUP BY
    s.service_id,
    s.service_name,
    s.service_category,
    s.service_price
),
ranked_services AS (
  SELECT
    *,
    DENSE_RANK() OVER (
    ORDER BY total_service_revenue DESC
  ) AS revenue_rank
  FROM service_summary
)
SELECT
  service_name,
  service_category,
  service_price,
  total_usage_records,
  total_quantity,
```

```sql
    unique_bookings,
    total_service_revenue,
    ROUND(
    total_service_revenue /
    NULLIF(total_usage_records, 0),
    2
  ) AS avg_revenue_per_usage,
    revenue_rank,
    CASE
    WHEN total_service_revenue >= 10000
      THEN 'High Revenue Service'
    WHEN total_service_revenue >= 5000
      THEN 'Medium Revenue Service'
    ELSE 'Low Revenue Service'
  END AS service_performance
  FROM ranked_services
  ORDER BY total_service_revenue DESC;
```

OUTPUT — 5 ROWS

service_name	service_category	service_price	total_usage_records	total_quantity
Breakfast	Food	500	7	17
Spa	Wellness	2000	4	4
Airport Transfer	Transport	1500	3	3
Room Service	Food	700	3	6
Laundry	Housekeeping	300	3	4

service_name	unique_bookings	total_service_revenue	avg_revenue_per_usage	revenue_rank	service_performance
Breakfast	7	8500	1214.29	1	Medium Revenue Service
Spa	4	8000	2000	2	Medium Revenue Service
Airport Transfer	3	4500	1500	3	Low Revenue Service
Room Service	3	4200	1400	4	Low Revenue Service
Laundry	3	1200	400	5	Low Revenue Service

KEY INSIGHT

Breakfast is the volume leader (17 units over seven bookings) and the top earner at ₹8,500, narrowly ahead of Spa at ₹8,000 from only four uses — Spa earns the highest revenue per use at ₹2,000. Services contribute ₹26,400 in total.

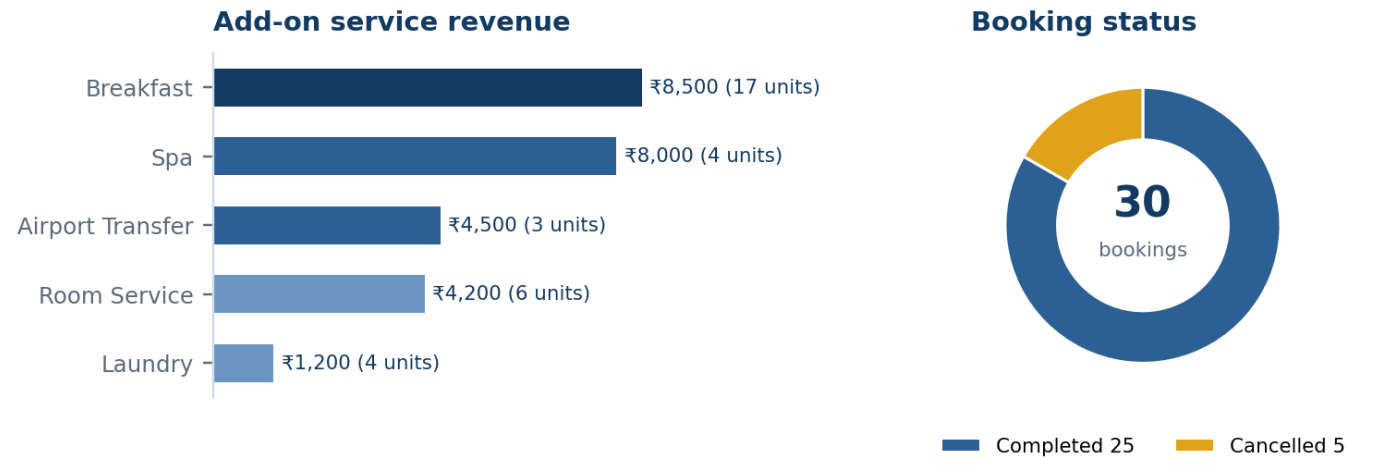


Figure 7.3 — Add-on service revenue (Question 13) and booking status split.

Question 14 — Hotel ranking and KPI comparison

Brings booking, rating and expense KPIs into one table per hotel, ranks hotels by completed revenue and assigns a performance category, with High Cancellation Risk taking priority over Strong Performance (rating 4.5 or more with cancellations under 15%) and Cost Improvement Needed.

QUESTION 14 — HOTEL RANKING & KPI COMPARISON

```sql
WITH booking_kpis AS (
  SELECT
    hotel_id,
    COUNT(*) AS total_bookings,
    SUM(CASE
      WHEN booking_status = 'Completed' THEN 1
      ELSE 0
      END) AS completed_bookings,
    SUM(CASE
      WHEN booking_status = 'Cancelled' THEN 1
      ELSE 0
      END) AS cancelled_bookings,
    SUM(CASE
      WHEN booking_status = 'Completed'
        THEN total_amount
      ELSE 0
      END) AS completed_revenue
  FROM bookings
  GROUP BY hotel_id
),
review_kpis AS (
  SELECT
    h.hotel_id,
    ROUND(AVG(rv.rating), 2) AS average_rating
  FROM hotels h
  LEFT JOIN rooms r
    ON h.hotel_id = r.hotel_id
  LEFT JOIN booking_rooms br
    ON r.room_id = br.room_id
  LEFT JOIN reviews rv
    ON br.booking_id = rv.booking_id
  GROUP BY h.hotel_id
),
expense_kpis AS (
  SELECT
    hotel_id,
    SUM(expense_amount) AS total_expenses
  FROM expenses
  GROUP BY hotel_id
),
hotel_performance AS (
  SELECT
    h.hotel_id,
    h.hotel_name,
    COALESCE(bk.total_bookings, 0) AS total_bookings,
    COALESCE(bk.completed_bookings, 0) AS completed_bookings,
    COALESCE(bk.cancelled_bookings, 0) AS cancelled_bookings,
    COALESCE(bk.completed_revenue, 0) AS completed_revenue,
    COALESCE(rk.average_rating, 0) AS average_rating,
    COALESCE(ek.total_expenses, 0) AS total_expenses,
    ROUND(
    COALESCE(bk.cancelled_bookings, 0) * 100.0 /
    NULLIF(bk.total_bookings, 0), 2
  ) AS cancellation_rate_pct,
    COALESCE(bk.completed_revenue, 0)
    - COALESCE(ek.total_expenses, 0)
    AS revenue_after_expenses_proxy
  FROM hotels h
  LEFT JOIN booking_kpis bk
    ON h.hotel_id = bk.hotel_id
  LEFT JOIN review_kpis rk
    ON h.hotel_id = rk.hotel_id
  LEFT JOIN expense_kpis ek
    ON h.hotel_id = ek.hotel_id
)
SELECT
  hotel_name,
  total_bookings,
  completed_bookings,
  cancelled_bookings,
  completed_revenue,
  cancellation_rate_pct,
  average_rating,
  total_expenses,
  revenue_after_expenses_proxy,
  DENSE_RANK() OVER (
  ORDER BY completed_revenue DESC
) AS revenue_rank,
  CASE
  WHEN cancellation_rate_pct >= 30
    THEN 'High Cancellation Risk'
  WHEN average_rating >= 4.5
    AND cancellation_rate_pct < 15
    THEN 'Strong Performance'
  WHEN revenue_after_expenses_proxy < 0
    THEN 'Cost Improvement Needed'
  ELSE 'Moderate Performance'
END AS performance_category
```

```
FROM hotel_performance
ORDER BY revenue_rank;
```

OUTPUT — 5 ROWS

hotel_name	total_bookings	completed_bookings	cancelled_bookings	completed_revenue	cancellation_rate_pct
Grand Palace Hotel	7	6	1	111000	14.29
Mountain Escape	5	5	0	80000	0
City Business Hotel	6	6	0	66000	0
Sea View Resort	7	4	3	48000	42.86
Royal Comfort Inn	5	4	1	22500	20

hotel_name	average_rating	total_expenses	revenue_after_expenses_proxy	revenue_rank	performance_category
Grand Palace Hotel	4.75	142000	-31000	1	**Strong Performance**
Mountain Escape	4.67	106000	-26000	2	**Strong Performance**
City Business Hotel	3.9	130000	-64000	3	**Cost Improvement Needed**
Sea View Resort	3.67	115000	-67000	4	**High Cancellation Risk**
Royal Comfort Inn	3.83	97000	-74500	5	**Cost Improvement Needed**

KEY INSIGHT

Grand Palace Hotel (₹111,000, rating 4.75) and Mountain Escape (₹80,000, rating 4.67, no cancellations) are the two Strong Performers. Sea View Resort is held back by its cancellation rate, while City Business Hotel and Royal Comfort Inn are marked for cost improvement.

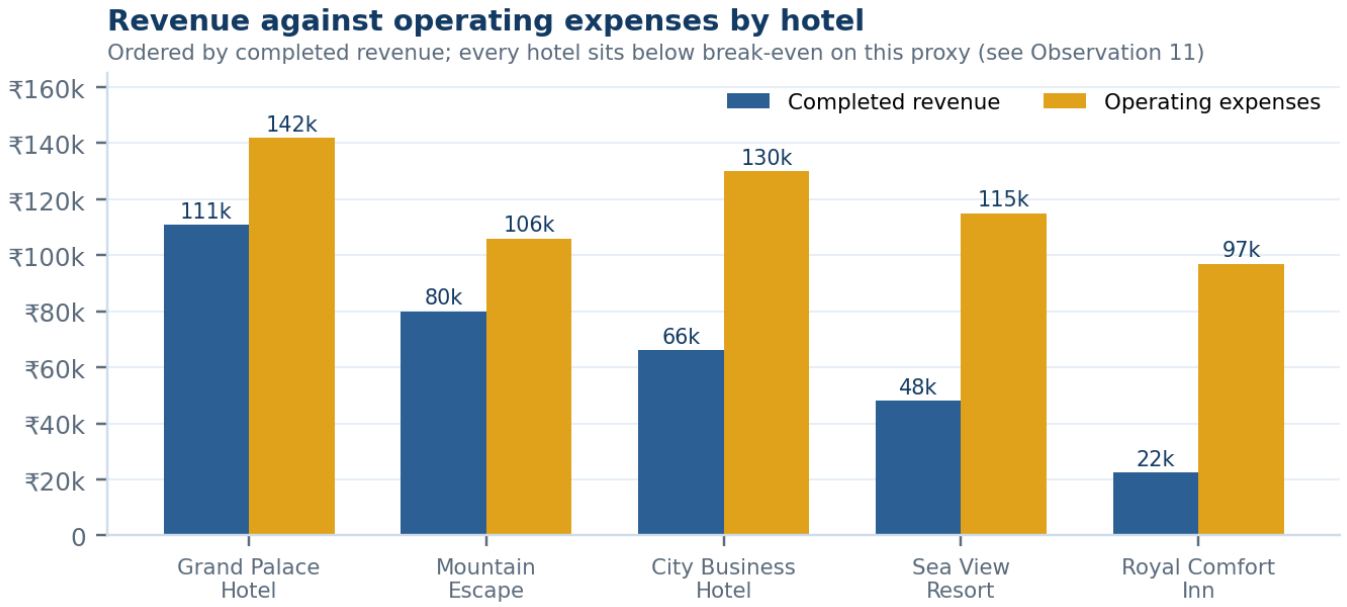


Figure 7.5 — Completed revenue and operating expenses by hotel (Question 14).

Question 15 — Final business performance summary

Five one-row CTEs — bookings, payments, reviews, expenses and services — are brought together with `CROSS JOIN` into a single group-level scorecard.

QUESTION 15 — FINAL HOTEL BUSINESS PERFORMANCE SUMMARY

```
USE hotel_analytics;
WITH booking_summary AS (
  SELECT
    COUNT(*) AS total_bookings,
    SUM(CASE
      WHEN booking_status = 'Completed' THEN 1
      ELSE 0
      END) AS completed_bookings,
    SUM(CASE
      WHEN booking_status = 'Cancelled' THEN 1
      ELSE 0
      END) AS cancelled_bookings,
    SUM(CASE
      WHEN booking_status = 'Completed'
        THEN total_amount
```

```sql
      ELSE 0
      END) AS completed_revenue
  FROM bookings
),
payment_summary AS (
  SELECT
    SUM(CASE
      WHEN payment_status = 'Paid'
        THEN payment_amount
      ELSE 0
      END) AS total_collected
  FROM payments
),
review_summary AS (
  SELECT
    ROUND(AVG(rating), 2) AS average_customer_rating,
    COUNT(*) AS total_reviews
  FROM reviews
),
expense_summary AS (
  SELECT
    SUM(expense_amount) AS total_operating_expenses
  FROM expenses
),
service_summary AS (
  SELECT
    SUM(total_cost) AS additional_service_revenue
  FROM service_usage
)
SELECT
  (SELECT COUNT(*) FROM hotels) AS total_hotels,
  bs.total_bookings,
  bs.completed_bookings,
  bs.cancelled_bookings,
  bs.completed_revenue,
  ROUND(
  bs.cancelled_bookings * 100.0 /
  NULLIF(bs.total_bookings, 0), 2
) AS cancellation_rate_pct,
  ps.total_collected,
  rs.average_customer_rating,
  rs.total_reviews,
  es.total_operating_expenses,
  ss.additional_service_revenue,
  bs.completed_revenue
  + COALESCE(ss.additional_service_revenue, 0)
  - COALESCE(es.total_operating_expenses, 0)
  AS revenue_after_expenses_proxy
FROM booking_summary bs
CROSS JOIN payment_summary ps
CROSS JOIN review_summary rs
CROSS JOIN expense_summary es
CROSS JOIN service_summary ss;
```

OUTPUT — 1 ROW (TRANSPOSED)

metric	value
total_hotels	5
total_bookings	30
completed_bookings	25
cancelled_bookings	5
completed_revenue	327500
cancellation_rate_pct	16.67
total_collected	311000
average_customer_rating	4.22
total_reviews	20
total_operating_expenses	590000
additional_service_revenue	26400
revenue_after_expenses_proxy	-236100

KEY INSIGHT

Across the group: 30 bookings, 25 completed and 5 cancelled (16.67%), ₹327,500 of completed gross revenue, ₹311,000 recorded as collected, ₹26,400 of add-on services and ₹590,000 of operating expenses, for a revenue-after-expenses proxy of −₹236,100. Guests rate the group 4.22 on average across 20 reviews.

SECTION 08

Code Review and Corrections

Six data and logic defects and six design observations found by checking the seeded data and queries against the business rules the schema implies, each with its correction.

Documenting what is wrong with your own work, and fixing it, says more about engineering judgement than a script that claims to be flawless. The issues below were found by running the script end to end and then testing the loaded data against the rules the schema implies: stays follow registration, payments follow bookings, cancellations are refunded, guests fit the room.

Defect 1 — Question 9 header comment is not a comment

SEVERITY: BLOCKING · STATUS: FIXED

The header of Question 9 begins with a single hyphen (`- QUESTION 9 ...`) instead of two. MySQL requires `--` (two hyphens and a space) to start a line comment, so the statement fails with ERROR 1064 (syntax error) at the first `-`, and in a script run without `--force` everything after it — Questions 9 to 15 — never executes.

CORRECTION — DEFECT 1

```
-- before
- QUESTION 9 — Hotel Operating Expenses & Cost Efficiency Analysis.

-- after
-- QUESTION 9 — Hotel Operating Expenses & Cost Efficiency Analysis.
```

Defect 2 — A cancelled booking has a Paid payment

SEVERITY: DATA INTEGRITY · STATUS: DOCUMENTED

Booking 26 is `Cancelled` and has a cancellation row with a ₹24,000 refund, yet payment 26 is recorded as `Paid` for ₹22,000. The four other cancelled bookings (6, 10, 16, 22) correctly carry a `Refunded` payment of 0. The stray payment inflates Grand Palace Hotel's collections to ₹126,000 against ₹104,000 of net revenue — the 121.15% collection rate in Question 1 — and adds ₹22,000 to the group total collected in Question 15.

CORRECTION — DEFECT 2

```
-- Detect: cancelled bookings that still show a Paid payment
SELECT b.booking_id, p.payment_amount, p.payment_status
FROM bookings b
JOIN payments p ON p.booking_id = b.booking_id
WHERE b.booking_status = 'Cancelled'
  AND p.payment_status = 'Paid';

-- Fix: align with the other cancelled bookings
UPDATE payments
SET payment_amount = 0,
    payment_status = 'Refunded'
WHERE booking_id = 26;
```

Defect 3 — Guests book before they are registered

SEVERITY: DATA INTEGRITY · STATUS: DOCUMENTED

In 19 of the 30 bookings (bookings 2 to 20) the `booking_date` is earlier than the guest's `registration_date` — for example guest 20 registered on 2025-06-05 but booked on 2025-04-05. A guest cannot book before the account exists. Dates are only compared in the checks that follow the load, so nothing in the schema catches it.

CORRECTION — DEFECT 3

```
-- Detect
SELECT b.booking_id, c.registration_date, b.booking_date
FROM bookings b
JOIN customers c ON c.customer_id = b.customer_id
WHERE b.booking_date < c.registration_date;

-- Fix: registration cannot be later than the guest's first booking
UPDATE customers c
JOIN (SELECT customer_id, MIN(booking_date) AS first_booking
      FROM bookings GROUP BY customer_id) f
  ON f.customer_id = c.customer_id
SET c.registration_date = LEAST(c.registration_date, f.first_booking);
```

Defect 4 — Booking 30 does not reconcile

SEVERITY: DATA INTEGRITY · STATUS: NEEDS SOURCE CHECK

For every other booking `total_amount` equals nightly rate × nights. Booking 30 has a nightly rate of ₹4,000 for 3 nights (₹12,000) but a `total_amount` of ₹18,000, and its pending payment (₹18,000) also exceeds the net value (₹17,000 after the ₹1,000 discount). Either the rate or the total is wrong; the source record must decide which.

CORRECTION — DEFECT 4

```sql
-- Detect: bookings whose total differs from rate x nights
SELECT b.booking_id, b.total_amount,
       br.nightly_rate * DATEDIFF(b.check_out_date, b.check_in_date) AS calculated
FROM bookings b
JOIN booking_rooms br ON br.booking_id = b.booking_id
WHERE b.total_amount <> br.nightly_rate * DATEDIFF(b.check_out_date, b.check_in_date);
```

Defect 5 — Party size exceeds room capacity

SEVERITY: DATA INTEGRITY · STATUS: DOCUMENTED

Four bookings exceed `room_types.max_guests`: booking 3 (3 guests in a 2-guest room), booking 6 (4 in 3), booking 20 (5 in 2) and booking 22 (3 in 2). The schema has the capacity stored but nothing enforces it.

CORRECTION — DEFECT 5

```sql
-- Detect
SELECT b.booking_id, b.number_of_guests, rt.max_guests
FROM bookings b
JOIN booking_rooms br ON br.booking_id = b.booking_id
JOIN rooms r ON r.room_id = br.room_id
JOIN room_types rt ON rt.room_type_id = r.room_type_id
WHERE b.number_of_guests > rt.max_guests;

-- Prevent: a BEFORE INSERT trigger on booking_rooms should compare the two and
-- raise SIGNAL SQLSTATE '45000' when the party does not fit.
```

Defect 6 — Refund equals the gross booking value

SEVERITY: LOGIC · STATUS: DOCUMENTED

`cancellations.refund_amount` equals the gross `total_amount` for all five cancellations (18,000, 9,000, 12,000, 12,000, 24,000), ignoring both the discount and what was actually paid. The matching payment rows show 0, so the refund ledger and the payment ledger disagree. Question 7 totals ₹75,000 of refunds that were never collected.

CORRECTION — DEFECT 6

```sql
-- Refund what was paid, not the gross price
UPDATE cancellations c
JOIN bookings b ON b.booking_id = c.booking_id
SET c.refund_amount = b.total_amount - b.discount_amount
WHERE c.refund_amount = b.total_amount;
-- (and record the corresponding Paid payment before it is Refunded)
```

Design observations

Observation 7 — Two incompatible occupancy measures

`hotels.total_rooms` advertises 65–120 rooms but only five per hotel exist in `rooms`. Question 1 divides room-nights by `total_rooms × 151` and reports 0.07–0.15% occupancy; Question 11 divides by the five modelled rooms and reports 80–100% utilisation. Load the full inventory, or derive `total_rooms` from `COUNT(rooms)`, and compute occupancy as room-nights sold ÷ (rooms × days in period).

Observation 8 — Dead column in Question 7

The `hotel_risk` CTE contains `SUM(DISTINCT 0) AS placeholder`, a leftover that is never selected. Remove it.

Observation 9 — Lead-time buckets do not discriminate

Lead times in the data run from 5 to 12 days, so Question 10 returns a single bucket. Tune the thresholds to the data (for example up to 7, 8–10, 11 or more) or load a wider spread of booking dates.

Observation 10 — marketing_campaigns is never used

The table is loaded but no query reads it and no column links a booking to a campaign, although Question 4 is titled “Booking Source & Marketing Channel”. Add `campaign_id` to `bookings`, or join on channel and date range, to measure campaign return.

Observation 11 — Revenue and cost are on different bases

Question 1 reports net revenue (total less discount) while Questions 2–14 use gross `total_amount`, so Grand Palace Hotel shows ₹104,000 in one and ₹111,000 in another. Expenses cover January–April only, and the staff postings (for example ₹90,000 for hotel 1) do not reconcile to the employee payroll (₹145,000 a month). Pick one revenue basis and reconcile payroll before reading the surplus metrics as profit.

Observation 12 — Constraints that would have caught the defects

`payments.transaction_reference` has no UNIQUE constraint (unlike a reference number in a ledger), `reviews` allows several reviews per booking, and there are no CHECK constraints on dates, amounts or ratings. Add `UNIQUE (transaction_reference)`, `UNIQUE (booking_id)` on reviews, and `CHECK (check_out_date > check_in_date)`, `CHECK (rating BETWEEN 1 AND 5)`, `CHECK (payment_amount >= 0)`.

CORRECTION — OBSERVATION 12 (CONSTRAINTS)

```
ALTER TABLE payments ADD CONSTRAINT uq_payment_ref UNIQUE (transaction_reference);
ALTER TABLE reviews ADD CONSTRAINT uq_review_booking UNIQUE (booking_id);
ALTER TABLE bookings ADD CONSTRAINT ck_stay CHECK (check_out_date > check_in_date);
ALTER TABLE reviews ADD CONSTRAINT ck_rating CHECK (rating BETWEEN 1 AND 5);
ALTER TABLE payments ADD CONSTRAINT ck_pay_amount CHECK (payment_amount >= 0);
```

Review Summary

Finding	Severity	Status	Action
Q9 header comment has a single hyphen	Blocking	Fixed	Use `--`
Cancelled booking 26 has a Paid payment	Data integrity	Documented	Set to Refunded / 0
19 bookings precede guest registration	Data integrity	Documented	Align registration dates
Booking 30 total ≠ rate × nights	Data integrity	Source check	Verify rate or total
4 bookings exceed room capacity	Data integrity	Documented	Validation trigger
Refund equals gross value, not amount paid	Logic	Documented	Refund net paid
Two occupancy measures disagree	Design	Roadmap	Model full inventory
Unused `placeholder` column in Q7	Cosmetic	Documented	Remove
Q10 buckets collapse to one	Design	Roadmap	Re-tune thresholds
marketing_campaigns unused	Design	Roadmap	Link bookings to campaigns
Gross vs net revenue, partial expenses	Design	Roadmap	One revenue basis
Missing UNIQUE and CHECK constraints	Design	Roadmap	Add constraints

SECTION

09 Results, Setup and Roadmap

What the project delivers, how to run it from a clean MySQL server, and where it goes next.

CHARTS	TABLES	ANALYTICAL QUERIES	REVIEW FINDINGS
6	15	15	12
revenue, channels, ratings	15 foreign keys	CTE-based	6 defects, 6 observations

Key Findings from the Dataset

Question	Answer from the data
Largest completed revenue	Grand Palace Hotel — ₹111,000 gross (₹104,000 net) from six completed bookings
Best guest satisfaction	Grand Palace Hotel — 4.75 average over six reviews; group average 4.22 over 20 reviews
Highest cancellation exposure	Sea View Resort — 3 of 7 bookings (42.86%), ₹42,000 of refunds
Top booking channel	Hotel Website — ₹89,000 completed revenue; the two commission-free direct channels earn 51.3% of the total
Weakest channel	Booking.com — 57.14% cancellation rate, ₹38,000 completed revenue after ₹5,700 commission
Best room type	Suite — ₹86,000 from six completed stays
Top add-on service	Breakfast — ₹8,500 from 17 units; Spa earns most per use at ₹2,000
Revenue trend	Six bookings a month; revenue peaks at ₹75,000 in February and April and dips to ₹56,500 in May
Most valuable guest	Rahul Verma — ₹36,000 over two stays, the only High Value guest
Group totals	30 bookings, 25 completed, 16.67% cancelled; ₹327,500 completed revenue; ₹26,400 add-on services; ₹590,000 expenses

Running the Project

The script assumes a clean MySQL 8.0 server with no existing `hotel_analytics` schema. It must be executed in order, because every foreign key is enforced at creation time. Correct the Question 9 comment first (Defect 1), otherwise the run stops there.

SETUP — COMMAND LINE

```
# 1. Execute the script end to end
mysql -u root -p < hotel_analytics.sql

# 2. Verify the build
mysql -u root -p -e "USE hotel_analytics; SHOW TABLES;"
mysql -u root -p -e "USE hotel_analytics; SELECT COUNT(*) FROM bookings;"

# 3. Run a single analysis
mysql -u root -p hotel_analytics < sql/05_analytics.sql
```

IF YOU ARE RE-RUNNING THE SCRIPT

Drop the schema first with `DROP DATABASE IF EXISTS hotel_analytics;`. Re-running over an existing schema produces ERROR 1050 on every `CREATE TABLE`.

Suggested Repository Structure

REPOSITORY LAYOUT

```
hotel-booking-analytics/
├── README.md                    # overview, ER diagram, setup, sample output
├── sql/
│   ├── 01_schema.sql            # Steps 1-17 : database and tables
│   ├── 02_seed_master.sql       # Steps 18-23: hotels ... customers
│   ├── 03_seed_operations.sql   # Steps 24-32: rooms ... expenses
│   ├── 04_checks.sql            # Steps 33-35: row counts and integrity
│   ├── 05_analytics.sql         # Questions 1-15
│   └── 06_fixes.sql             # corrections from Section 08
├── docs/
│   ├── er-diagram.png
│   └── project-documentation.pdf # this document
└── screenshots/                 # query output from MySQL Workbench
```

Complete Object Inventory

Type	Count	Objects
Database	1	hotel_analytics
Tables	15	customers, hotels, room_types, rooms, bookings, booking_rooms, payments, cancellations, reviews, services, service_usage, employees, expenses, marketing_campaigns, booking_sources
Foreign keys	15	rooms (2), bookings (3), booking_rooms (2), payments, cancellations, reviews (2), service_usage (2), employees, expenses
Analytical queries	15	Questions 1–15: KPI report, profitability, trend, channels, guest value, room types, cancellations, reviews, costs, lead time, utilisation, collections, services, ranking, summary
Integrity checks	2	Invalid date range and orphan-booking checks after the load

Roadmap

Enhancement	Detail
Fix the data defects	Apply the Section 08 corrections and add the missing UNIQUE and CHECK constraints so the database rejects them in future.
Views	Turn the repeated joins (booking to room to hotel, booking to payment) into named views such as `booking_summary` and `hotel_revenue_summary`, so Questions 2–14 read as short statements.
Stored procedures	Wrap the guest-level lookups — bookings, payments, reviews — in procedures that take a `customer_id`, and a transactional `CreateBooking` that writes booking, room and payment atomically.
Full room inventory	Load every advertised room and compute true occupancy as room-nights sold divided by room-nights available.
Campaign attribution	Link bookings to `marketing_campaigns` and report return on campaign budget alongside channel commission.
Indexing	Add composite indexes such as `bookings(hotel_id, booking_date)` and `payments(booking_id)` and compare `EXPLAIN` output before and after.
Revenue management	Derive dynamic-price recommendations from lead time, occupancy and cancellation risk by room type.
Visual layer	Point Power BI or Tableau at the reporting queries to turn Sections 06 and 07 into an interactive hotel dashboard.

Skills Demonstrated

Taken together, this project covers the working vocabulary of relational database development and analytics: schema design and normalisation, referential integrity, schema evolution with `ALTER TABLE`, multi-step CTE pipelines, conditional aggregation, window functions, null-safe ratio maths, date arithmetic, rule-based segmentation — and, in Section 08, the habit of testing your own data against business rules and documenting what you would change.

E N D O F D O C U M E N T

hotel_analytics — 15 tables, 15 foreign keys, 15 analytical questions, 239 seeded rows. Every statement in the original script is reproduced in this document, together with the output it returns and the corrections applied.